HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

TIỂU LUẬN

Giảng viên hướng dẫn	**: PGS.TS. Trần Đình Quế**
Họ và tên sinh viên	**: Nguyễn Quang Minh**
Mã sinh viên	**: B23DCCN557**
Nhóm học phần	**: 05**

Hà Nội – 2026

MỤC LỤC

PHẦN MỞ ĐẦU 4
1. Bối cảnh và tính cấp thiết của đề tài 4
2. Mục tiêu nghiên cứu và phạm vi đề tài 5
3. Phương pháp luận và cách tiếp cận nghiên cứu 6
4. Ý nghĩa khoa học và giá trị thực tiễn 7
5. Cấu trúc của báo cáo tiểu luận 7
CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN TRÍ TUỆ NHÂN TẠO 8
1.1. Giai đoạn đặt nền móng: trước năm 1956 9
1.2. Năm 1956: AI trở thành một lĩnh vực nghiên cứu 9
1.3. Giai đoạn phát triển ban đầu: cuối thập niên 1950 đến đầu thập niên 1970 10
1.4. "Mùa đông AI" thứ nhất: khoảng thập niên 1970 11
1.5. Sự phát triển của hệ chuyên gia: thập niên 1970–1980 12
1.6. Sự phát triển của học máy và mạng nơ-ron: thập niên 1980–2000 12
1.7. Sự bùng nổ của học sâu: từ khoảng năm 2010 13
1.8. Transformer và sự phát triển của mô hình ngôn ngữ lớn 14
1.9. AI tạo sinh và sự phổ biến của AI trong đời sống 14
1.10. Bảng tổng hợp các dấu mốc quan trọng 15
CHƯƠNG 2. CÁC KỸ THUẬT HỌC MÁY CƠ BẢN 16
2.1. Tổng quan về học máy 16
2.2. Các thành phần cơ bản của một mô hình ML 18
2.3. Chuẩn bị thư viện cho các đoạn code 19
2.4. Hồi quy tuyến tính — Linear Regression 20
2.5. Hồi quy logistic — Logistic Regression 24
2.6. K láng giềng gần nhất — K-Nearest Neighbors 28
2.7. Máy vector hỗ trợ tuyến tính — Linear SVM 31
CHƯƠNG 3.CNN 35
1.Tổng quan kiến thức CNN 35
2.Giới thiệu và dataset 36
a. Fashion MNIST 36
b. Kuzushiji-MNIST 38
3. Các mô hình CNN 41
3.1. CNN cơ bản – Mạng tích chập nền tảng cho phân loại hình ảnh 41
3.2. VGG16 – Mạng tích chập sâu với các bộ lọc nhỏ 41
3.3. ResNet50 – Mạng tích chập sâu sử dụng học phần dư 41

4.Thực nghiệm trên các tập dataset 42
4.1. Dataset 1 42
4.2. Dataset 2 51
CHƯƠNG 4. RNN 60
1.Tổng quan kiến thức về RNN 60
2.Giới thiệu các dataset 61
a. SJC Gold Price Vietnam 61
b. Amazon Stock Price (AMZN) 62
3. Các mô hình huấn luyện RNN 64
3.1. RNN cơ bản — Recurrent Neural Network 64
3.2. LSTM — Long Short-Term Memory 65
3.3. Stacked LSTM — LSTM nhiều tầng 65
4.Thực nghiệm trên các tập dataset 65
4.1. Dataset 1 65
4.2 Dataset 2 73

PHẦN MỞ ĐẦU

1. Bối cảnh và tính cấp thiết của đề tài

Trí tuệ nhân tạo (Artificial Intelligence – AI) là một lĩnh vực của khoa học máy tính, hướng đến xây dựng các hệ thống có khả năng thực hiện những nhiệm vụ liên quan đến nhận biết, học hỏi, suy luận và ra quyết định. Sự phát triển của dữ liệu số, năng lực tính toán và các phương pháp xử lý thông tin đã tạo điều kiện để AI được nghiên cứu và ứng dụng trong nhiều lĩnh vực. Những bài toán như nhận dạng hình ảnh, dự báo chuỗi thời gian, tìm kiếm thông tin và hỗ trợ giải quyết nhiệm vụ cho thấy vai trò của AI trong việc khai thác dữ liệu và xây dựng các hệ thống thông minh.

Quá trình phát triển của AI gắn liền với nhiều hướng tiếp cận khác nhau. Các phương pháp dựa trên luật tập trung biểu diễn kiến thức và xây dựng cơ chế suy luận. Các phương pháp học máy khai thác dữ liệu để tìm ra quy luật, trong khi học sâu sử dụng mạng nơ-ron nhiều tầng để học những biểu diễn phù hợp với nhiệm vụ. Những hướng tiếp cận này có thể bổ sung cho nhau tùy theo yêu cầu của bài toán. Do đó, việc tìm hiểu AI cần được đặt trong một hệ thống kiến thức có liên kết, từ bối cảnh lịch sử đến nguyên lý hoạt động và khả năng ứng dụng.

Học máy (Machine Learning – ML) là nền tảng quan trọng trong nhiều phương pháp AI. Thông qua dữ liệu và quá trình tối ưu hóa, mô hình có thể học cách phân loại, dự đoán hoặc phát hiện cấu trúc trong dữ liệu. Tuy nhiên, hiệu quả của mô hình chịu ảnh hưởng bởi nhiều yếu tố như chất lượng dữ liệu, cách lựa chọn đặc trưng, phương pháp chia tập và tiêu chí đánh giá. Vì vậy, hiểu các kỹ thuật ML cơ bản là điều kiện cần thiết để xây dựng mô hình và diễn giải kết quả một cách có căn cứ.

Trong học sâu, mạng nơ-ron tích chập (Convolutional Neural Network – CNN) và mạng nơ-ron hồi quy (Recurrent Neural Network – RNN) là hai kiến trúc tiêu biểu. CNN khai thác các đặc trưng cục bộ và cơ chế chia sẻ trọng số, phù hợp với nhiều bài toán xử lý hình ảnh. RNN xử lý thông tin theo trình tự thông qua trạng thái ẩn, phù hợp với dữ liệu chuỗi. Việc nghiên cứu hai kiến trúc này giúp người học hiểu mối liên hệ giữa cấu trúc dữ liệu và thiết kế mô hình, đồng thời tiếp cận những vấn đề như quá khớp, tối ưu hóa và khả năng khái quát hóa.

Bên cạnh khả năng học từ dữ liệu, một hệ thống AI còn có thể cần tổ chức và sử dụng tri thức từ nhiều nguồn. Đồ thị tri thức (Knowledge Graph) biểu diễn thông tin thông qua thực thể và quan hệ, hỗ trợ liên kết và khai thác kiến thức trong ngữ cảnh. Cách tổ chức này có ý nghĩa đối với những bài toán mà mối quan hệ giữa các đối tượng là thành phần quan trọng của thông tin.

Sinh tăng cường truy xuất (Retrieval-Augmented Generation – RAG) kết hợp việc truy xuất thông tin với quá trình sinh nội dung. Thông tin được truy xuất cung cấp thêm căn cứ cho mô hình khi thực hiện các nhiệm vụ cần kiến thức. Hướng tiếp cận này mở rộng góc nhìn từ việc xây dựng một mô hình đơn lẻ sang thiết kế hệ thống có khả năng sử dụng nguồn dữ liệu bên ngoài.

Đối với dữ liệu có cấu trúc quan hệ, học sâu trên đồ thị tạo điều kiện khai thác đồng thời đặc trưng của đối tượng và các liên kết giữa chúng. Mạng nơ-ron đồ thị mở rộng khả năng học biểu diễn sang những dữ liệu mà cấu trúc kết nối có ảnh hưởng đến kết quả xử lý. Đây là cơ

sở để tiểu luận nghiên cứu nội dung Deep Graph và tìm hiểu cách học sâu tiếp cận dữ liệu đồ thị.

Ở cấp độ tổ chức hệ thống, hệ đa Agent hướng đến việc phối hợp nhiều tác tử để thực hiện nhiệm vụ. Các tác tử có thể đảm nhận những vai trò khác nhau, trao đổi thông tin và sử dụng công cụ theo cơ chế được thiết kế. Nghiên cứu hướng tiếp cận này giúp làm rõ những vấn đề về phân công, điều phối và kiểm soát quá trình giải quyết nhiệm vụ trong hệ thống AI.

Từ những cơ sở trên, việc nghiên cứu các nền tảng và hướng tiếp cận trong AI có ý nghĩa đối với quá trình học tập chuyên ngành công nghệ thông tin. Người học cần hiểu nguyên lý, điều kiện áp dụng và giới hạn của từng phương pháp để lựa chọn giải pháp phù hợp. Tiểu luận được thực hiện nhằm hệ thống hóa kiến thức từ lịch sử phát triển AI đến học máy, học sâu, khai thác tri thức và phối hợp tác tử, qua đó xây dựng nền tảng cho việc nghiên cứu và triển khai các ứng dụng thông minh.

2. Mục tiêu nghiên cứu và phạm vi đề tài

2.1. Mục tiêu tổng quát

Mục tiêu tổng quát của tiểu luận là nghiên cứu và hệ thống hóa những kiến thức nền tảng về trí tuệ nhân tạo, đồng thời tìm hiểu các phương pháp và kiến trúc tiêu biểu phục vụ xử lý dữ liệu, khai thác tri thức và tổ chức hệ thống thông minh. Nội dung được triển khai theo trình tự từ cơ sở lịch sử và lý thuyết đến phân tích phương pháp và minh họa thực nghiệm.

Thông qua quá trình nghiên cứu, tiểu luận hướng đến hình thành khả năng nhận diện đặc điểm của bài toán, lựa chọn phương pháp phù hợp và đánh giá kết quả dựa trên các tiêu chí rõ ràng. Đồng thời, tiểu luận giúp người học hiểu mối liên hệ giữa dữ liệu, mô hình, nguồn tri thức và cơ chế phối hợp trong một hệ thống AI.

2.2. Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, tiểu luận xác định các mục tiêu cụ thể sau:

- Hệ thống hóa lịch sử phát triển AI: Tìm hiểu bối cảnh hình thành, các giai đoạn phát triển và những hướng tiếp cận tiêu biểu; làm rõ vai trò của thuật toán, dữ liệu, tri thức và năng lực tính toán trong sự phát triển của lĩnh vực.
- Nghiên cứu các kỹ thuật ML cơ bản: Trình bày những khái niệm nền tảng, các hình thức học và phương pháp tiêu biểu; phân tích quy trình xử lý dữ liệu, xây dựng mô hình và đánh giá kết quả.
- Tìm hiểu kiến trúc CNN và RNN: Làm rõ nguyên lý hoạt động, cấu trúc và quá trình học; phân tích khả năng ứng dụng của các kiến trúc này đối với dữ liệu hình ảnh và dữ liệu chuỗi.
- Nghiên cứu Knowledge Graph và RAG: Tìm hiểu cách tổ chức tri thức, cơ chế truy xuất và việc sử dụng thông tin bên ngoài để hỗ trợ xử lý nhiệm vụ; nhận diện vai trò của chất lượng nguồn dữ liệu đối với kết quả đầu ra.
- Tìm hiểu Deep Graph và hệ đa Agent: Phân tích cách học sâu khai thác cấu trúc đồ thị và cách nhiều tác tử phối hợp thực hiện nhiệm vụ; làm rõ các thành phần và vấn đề cần chú ý khi thiết kế hệ thống.

- Kết hợp lý thuyết với thực hành và đánh giá: Triển khai các ví dụ phù hợp, trực quan hóa kết quả và so sánh phương pháp theo những tiêu chí như sai số, khả năng khái quát hóa và chi phí tính toán.

2.3. Phạm vi nghiên cứu

Về nội dung, tiểu luận tập trung vào bảy nhóm chủ đề: lịch sử phát triển AI, các kỹ thuật ML cơ bản, CNN, RNN, Knowledge Graph và RAG, Deep Graph, hệ đa Agent. Mỗi nhóm được nghiên cứu ở mức độ phù hợp với mục tiêu môn học, chú trọng khái niệm, nguyên lý, cấu trúc và khả năng ứng dụng.

Về lý thuyết, trọng tâm nghiên cứu gồm biểu diễn dữ liệu, cơ chế học và suy luận, thiết kế kiến trúc, quy trình thực hiện và phương pháp đánh giá. Trong phạm vi tiểu luận, Deep Graph được hiểu là học sâu trên dữ liệu đồ thị, với mạng nơ-ron đồ thị là một nhóm phương pháp trọng tâm. Knowledge Graph và RAG được xem xét theo chức năng của từng hướng tiếp cận và khả năng kết hợp khi phù hợp với yêu cầu hệ thống.

Về thực nghiệm, các ví dụ được lựa chọn nhằm minh họa nguyên lý và quy trình triển khai. Những bài toán như phân loại hình ảnh và dự báo chuỗi thời gian giúp làm rõ ảnh hưởng của dữ liệu, kiến trúc và tham số huấn luyện. Kết quả được phân tích trong điều kiện cụ thể của tập dữ liệu và môi trường thực thi, tạo cơ sở cho việc nhận xét ưu điểm, hạn chế và khả năng ứng dụng của phương pháp.

3. Phương pháp luận và cách tiếp cận nghiên cứu

Tiểu luận tiếp cận vấn đề theo hướng kết hợp nghiên cứu lý thuyết, phân tích phương pháp và minh họa thực nghiệm. Các nội dung được tổ chức từ nền tảng đến mở rộng, giúp kiến thức của chương trước hỗ trợ việc tìm hiểu chương sau. Cách tiếp cận này tạo sự thống nhất giữa các nhóm chủ đề và làm rõ vai trò của từng kỹ thuật trong quá trình xây dựng hệ thống AI.

Phương pháp nghiên cứu tài liệu được sử dụng để thu thập và hệ thống hóa kiến thức từ giáo trình, bài báo khoa học và tài liệu chính thức. Các nguồn được lựa chọn dựa trên mức độ phù hợp với chủ đề và độ rõ ràng của nội dung. Quá trình đối chiếu tài liệu giúp thống nhất khái niệm, ký hiệu và cách diễn giải, đồng thời hỗ trợ phân biệt những phương pháp có tên gọi hoặc chức năng gần nhau.

Phương pháp phân tích và so sánh được áp dụng để làm rõ nguyên lý, điều kiện sử dụng và giới hạn của từng hướng tiếp cận. Các phương pháp được xem xét theo dạng dữ liệu, cơ chế xử lý thông tin, yêu cầu huấn luyện và khả năng đánh giá. Việc so sánh được đặt trong bối cảnh bài toán cụ thể, bởi một kiến trúc có nhiều tầng hoặc nhiều thành phần hơn cần được kiểm chứng về hiệu quả thay vì chỉ đánh giá dựa trên mức độ phức tạp.

Đối với các phần thực nghiệm, quy trình gồm khảo sát dữ liệu, tiền xử lý, chia tập, xây dựng mô hình, huấn luyện và đánh giá. Python cùng các thư viện như PyTorch và Keras hỗ trợ chuyển nguyên lý lý thuyết thành chương trình có thể thực thi. Việc chia dữ liệu được thực hiện phù hợp với đặc điểm bài toán; đối với chuỗi thời gian, thứ tự thời gian và giới hạn sử dụng thông tin tương lai cần được bảo đảm.

Kết quả được trình bày bằng bảng số liệu và biểu đồ trực quan. Các chỉ số đánh giá được lựa chọn theo nhiệm vụ, kết hợp với việc xem xét thời gian huấn luyện và phương pháp tham

chiếu. Quá trình phân tích chú trọng sự phù hợp giữa số liệu và kết luận, giúp đánh giá cả chất lượng đầu ra lẫn chi phí thực hiện. Đối với những nội dung ở cấp độ hệ thống, việc phân tích còn xem xét luồng thông tin, sự tương tác và khả năng kiểm soát các thành phần.

4. Ý nghĩa khoa học và giá trị thực tiễn

Về ý nghĩa khoa học, tiểu luận góp phần hệ thống hóa các kiến thức thuộc nhiều hướng tiếp cận trong trí tuệ nhân tạo. Việc đặt lịch sử phát triển AI, học máy, học sâu, biểu diễn tri thức và hệ tác tử trong cùng một bố cục giúp làm rõ mối liên hệ giữa các chủ đề. Trong phạm vi học tập, giá trị của tiểu luận nằm ở việc tổng hợp, phân tích và diễn giải kiến thức theo một trình tự nhất quán.

Tiểu luận cũng hỗ trợ hình thành tư duy nghiên cứu dựa trên bằng chứng. Một phương pháp cần được đánh giá thông qua dữ liệu, điều kiện thực nghiệm và tiêu chí so sánh phù hợp. Việc quan sát kết quả chưa đạt kỳ vọng có thể giúp nhận diện giới hạn của mô hình, ảnh hưởng của tham số hoặc đặc điểm của dữ liệu. Cách nhìn này tạo nền tảng để đề xuất cải tiến có cơ sở và tránh đưa ra những kết luận vượt quá phạm vi thử nghiệm.

Về giá trị thực tiễn, quá trình thực hiện giúp rèn luyện kỹ năng đọc tài liệu, xử lý dữ liệu, lập trình, xây dựng mô hình và trình bày kết quả. Những kỹ năng này có thể được vận dụng trong các nhiệm vụ như nhận dạng hình ảnh, dự báo dữ liệu theo thời gian, truy xuất thông tin và hỗ trợ người dùng. Việc tìm hiểu các công cụ triển khai cũng giúp người học tiếp cận quy trình phát triển ứng dụng AI từ dữ liệu đầu vào đến kết quả đầu ra.

Bên cạnh đó, các nội dung về tri thức, đồ thị và hệ đa Agent mở rộng tư duy từ lựa chọn thuật toán sang thiết kế hệ thống. Người học có cơ sở xem xét cách tổ chức dữ liệu, cung cấp thông tin cho mô hình, phân công nhiệm vụ và kiểm soát sự tương tác giữa các thành phần. Đây là những kiến thức cần thiết khi tiếp tục nghiên cứu hoặc xây dựng các ứng dụng có nhiều chức năng và nguồn dữ liệu.

5. Cấu trúc của báo cáo tiểu luận

Ngoài phần mở đầu, báo cáo được tổ chức thành bảy chương, triển khai theo trình tự từ kiến thức nền tảng đến các hướng tiếp cận mở rộng.

Chương 1: Lịch sử phát triển AI. Chương này trình bày bối cảnh hình thành, các giai đoạn phát triển và những hướng tiếp cận tiêu biểu của trí tuệ nhân tạo. Nội dung tạo cơ sở để hiểu sự thay đổi của phương pháp giải quyết vấn đề và vai trò của dữ liệu, thuật toán, tri thức cùng năng lực tính toán.

Chương 2: Các kỹ thuật ML cơ bản. Chương này giới thiệu các khái niệm nền tảng, hình thức học và kỹ thuật tiêu biểu. Đồng thời, chương trình bày quy trình xây dựng mô hình, xử lý dữ liệu, lựa chọn tham số và đánh giá kết quả, làm nền tảng cho các kiến trúc học sâu ở những chương tiếp theo.

Chương 3: CNN. Chương này nghiên cứu nguyên lý mạng nơ-ron tích chập, các thành phần kiến trúc và cơ chế học đặc trưng. Các ví dụ và thực nghiệm hỗ trợ phân tích khả năng xử lý dữ liệu hình ảnh, đồng thời cung cấp cơ sở để so sánh hiệu quả giữa những mô hình được lựa chọn.

Chương 4: RNN. Chương này tập trung vào cơ chế xử lý dữ liệu chuỗi, trạng thái ẩn và quá trình huấn luyện mạng nơ-ron hồi quy. Nội dung đề cập các biến thể và những khó khăn trong quá trình học, kết hợp minh họa bằng bài toán dự báo hoặc phân tích thông tin theo trình tự.

Chương 5: Knowledge Graph và RAG. Chương này tìm hiểu cách tổ chức tri thức thông qua thực thể và quan hệ, đồng thời trình bày cơ chế kết hợp truy xuất thông tin với sinh nội dung. Trọng tâm là vai trò của nguồn tri thức, quy trình khai thác dữ liệu và các yếu tố ảnh hưởng đến độ tin cậy của kết quả.

Chương 6: Deep Graph. Chương này trình bày học sâu trên dữ liệu đồ thị, bao gồm cách biểu diễn đối tượng, liên kết và khai thác thông tin từ cấu trúc quan hệ. Nội dung mở rộng kiến thức học sâu sang những bài toán mà sự kết nối giữa các đối tượng có ảnh hưởng đến nhiệm vụ xử lý.

Chương 7: Hệ đa Agent. Chương này nghiên cứu khái niệm tác tử, tổ chức vai trò, phân công nhiệm vụ và cơ chế tương tác trong hệ thống nhiều tác tử. Nội dung làm rõ khả năng phối hợp giữa các thành phần, đồng thời xem xét các vấn đề về điều phối, kiểm soát và đánh giá quá trình thực hiện nhiệm vụ.

CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN TRÍ TUỆ NHÂN TẠO

Trí tuệ nhân tạo, hay Artificial Intelligence (AI), là lĩnh vực nghiên cứu và phát triển các hệ thống máy tính có khả năng thực hiện những nhiệm vụ thường đòi hỏi trí thông minh của con người, như học tập, suy luận, nhận dạng, xử lý ngôn ngữ và ra quyết định.

AI phát triển qua nhiều thập kỷ, trải qua các giai đoạn đặt nền móng, đạt thành công ban đầu, gặp khó khăn rồi phục hồi bằng những phương pháp mới. Quá trình này cho thấy sự thay đổi

trong cách xây dựng hệ thống thông minh: từ việc con người viết các quy tắc để máy thực hiện, đến việc máy học quy luật từ dữ liệu và tạo ra nội dung mới.

1.1. Giai đoạn đặt nền móng: trước năm 1956

Trước khi thuật ngữ "trí tuệ nhân tạo" xuất hiện, các nhà khoa học đã nghiên cứu khả năng biểu diễn hoạt động tư duy bằng toán học và cơ chế tính toán. Hai câu hỏi quan trọng được đặt ra là: hoạt động của bộ não có thể được mô hình hóa hay không, và máy tính có thể thực hiện những hành vi được xem là thông minh hay không?

Mô hình nơ-ron nhân tạo năm 1943

Năm 1943, Warren McCulloch và Walter Pitts công bố một mô hình toán học đơn giản của nơ-ron thần kinh. Mô hình sử dụng các đầu vào và một điều kiện kích hoạt để xác định đầu ra, cho thấy một mạng gồm những phần tử đơn giản có thể thực hiện các phép toán logic. Đây là một trong những nền tảng lý thuyết quan trọng của mạng nơ-ron nhân tạo. Springer Nature Link

Ý nghĩa của công trình này nằm ở việc đưa hoạt động thần kinh vào phạm vi nghiên cứu toán học. Thay vì chỉ quan sát bộ não dưới góc độ sinh học, các nhà khoa học bắt đầu nghiên cứu cách xây dựng những hệ thống tính toán lấy cảm hứng từ bộ não.

Tuy nhiên, mô hình nơ-ron lúc đó còn đơn giản và chưa có khả năng học như các mạng nơ-ron hiện đại. Khoảng cách giữa một mô hình lý thuyết và một hệ thống thông minh thực tế vẫn rất lớn.

Alan Turing và câu hỏi về trí thông minh của máy

Năm 1950, Alan Turing công bố bài báo *Computing Machinery and Intelligence*, thảo luận khả năng máy tính thể hiện hành vi thông minh. Ông đề xuất một cách tiếp cận dựa trên trò chơi bắt chước, về sau thường được gọi là phép thử Turing. hec.edu

Trong cách diễn giải phổ biến của phép thử này, một người đánh giá giao tiếp bằng văn bản với một người và một máy tính mà không biết danh tính của hai bên. Nếu người đánh giá khó phân biệt bên nào là máy, hệ thống được xem là có khả năng mô phỏng hành vi hội thoại của con người trong điều kiện thử nghiệm.

Đóng góp của Turing là chuyển một câu hỏi mang tính triết học sang một vấn đề có thể nghiên cứu thông qua hành vi quan sát được. Tuy vậy, khả năng trò chuyện giống con người không tự động chứng minh rằng máy có ý thức hoặc hiểu thế giới giống con người.

1.2. Năm 1956: AI trở thành một lĩnh vực nghiên cứu

Một dấu mốc quan trọng trong lịch sử AI là chương trình nghiên cứu mùa hè tại Dartmouth College, Hoa Kỳ, năm 1956.

Bản đề xuất cho chương trình được viết năm 1955 bởi John McCarthy, Marvin Minsky, Nathaniel Rochester và Claude Shannon. Thuật ngữ *Artificial Intelligence* xuất hiện trong bản đề xuất này; chương trình Dartmouth năm 1956 thường được xem là sự kiện đánh dấu sự hình thành AI như một lĩnh vực nghiên cứu riêng. www-formal.stanford.edu

Ý tưởng trung tâm của chương trình là nghiên cứu liệu các khía cạnh của học tập và trí thông minh có thể được mô tả đủ chính xác để máy tính mô phỏng hay không.

Từ đây, nghiên cứu AI tập trung vào các nhiệm vụ như:

- Biểu diễn tri thức để máy tính có thể xử lý.
- Suy luận và giải quyết vấn đề.
- Học từ kinh nghiệm.
- Xử lý ngôn ngữ tự nhiên.
- Nhận dạng và tương tác với môi trường.

Ý nghĩa của sự kiện Dartmouth nằm ở việc tạo ra một định hướng nghiên cứu chung, kết nối các ý tưởng trước đó về logic, tính toán, học tập và mô hình thần kinh.

1.3. Giai đoạn phát triển ban đầu: cuối thập niên 1950 đến đầu thập niên 1970

Sau Dartmouth, AI bước vào thời kỳ nhiều kỳ vọng. Các nhà nghiên cứu xây dựng những chương trình chứng minh định lý, chơi trò chơi, xử lý ngôn ngữ và giải quyết vấn đề trong môi trường đơn giản.

Hai hướng tiếp cận đáng chú ý trong giai đoạn này là **AI dựa trên ký hiệu** và **các mô hình học lấy cảm hứng từ nơ-ron**.

AI dựa trên ký hiệu

AI dựa trên ký hiệu, còn gọi là *Symbolic AI*, biểu diễn thông tin bằng các ký hiệu và sử dụng quy tắc để suy luận.

Ví dụ, hệ thống có thể lưu hai thông tin:

- Mọi sinh viên trong lớp A đều học môn lập trình.
- Minh là sinh viên trong lớp A.

Từ đó, hệ thống suy ra Minh học môn lập trình.

Trong cách tiếp cận này, con người phải xác định cách biểu diễn thông tin và xây dựng các quy tắc xử lý. Máy tính áp dụng các quy tắc để tìm ra kết quả.

Ưu điểm của phương pháp là quá trình suy luận có thể rõ ràng và dễ kiểm tra. Tuy nhiên, việc mô tả đầy đủ tri thức thực tế bằng các quy tắc rất khó, đặc biệt khi thông tin thiếu, mơ hồ hoặc có ngoại lệ.

Perceptron và khả năng học từ dữ liệu

Năm 1958, Frank Rosenblatt công bố công trình về Perceptron, một mô hình có khả năng điều chỉnh trọng số để học cách phân loại dữ liệu. Đây là một bước tiến quan trọng so với những mô hình chỉ thực hiện các quy tắc cố định. PubMed

Có thể hiểu đơn giản rằng Perceptron nhận đầu vào, tính toán kết quả và điều chỉnh các trọng số khi dự đoán sai. Qua nhiều ví dụ, mô hình dần học được ranh giới phân loại.

Tuy nhiên, Perceptron một tầng chỉ biểu diễn được các ranh giới phân loại tuyến tính. Hạn chế này khiến nó không giải quyết được một số bài toán đơn giản nhưng không phân tách tuyến tính, như XOR.

Các hệ thống tiêu biểu

Năm 1965, DENDRAL được phát triển để hỗ trợ suy luận về cấu trúc phân tử từ dữ liệu hóa học. Hệ thống cho thấy tri thức chuyên ngành có thể giúp máy tính giải quyết một nhiệm vụ cụ thể.

Năm 1966, Joseph Weizenbaum hoàn thành ELIZA, một chương trình hội thoại sử dụng các mẫu câu và quy tắc thay thế từ khóa. ELIZA tạo được cảm giác đang trò chuyện với người dùng, nhưng khả năng phản hồi chủ yếu dựa trên mẫu có sẵn. Computer History Museum

Những thành công ban đầu chứng minh rằng máy tính có thể thực hiện một số nhiệm vụ từng được xem là gắn liền với trí thông minh con người. Tuy nhiên, hiệu quả trong môi trường giới hạn chưa đồng nghĩa với khả năng hoạt động tốt trong thế giới thực.

1.4. “Mùa đông AI” thứ nhất: khoảng thập niên 1970

Sau thời kỳ kỳ vọng cao, AI gặp nhiều khó khăn. Những hệ thống hoạt động tốt với bài toán nhỏ thường không mở rộng được sang các tình huống phức tạp.

“Mùa đông AI” là cách gọi những giai đoạn sự quan tâm và nguồn tài trợ dành cho AI suy giảm do kết quả thực tế không đáp ứng kỳ vọng. Mốc thời gian có thể khác nhau giữa các tài liệu; đợt suy giảm đầu tiên thường được gắn với thập niên 1970.

Tại Anh, báo cáo Lighthill năm 1973 phê bình những hạn chế của nghiên cứu AI và góp phần làm giảm sự hỗ trợ dành cho lĩnh vực này. Artificial Intelligence Committee

Các khó khăn chính gồm:

- **Năng lực tính toán hạn chế:** máy tính chưa đủ mạnh để xử lý nhiều bài toán phức tạp.
- **Thiếu dữ liệu:** dữ liệu số hóa phục vụ nghiên cứu còn ít.
- **Bùng nổ tổ hợp:** số phương án cần xem xét tăng nhanh khi bài toán lớn hơn.
- **Khó biểu diễn tri thức thực tế:** các quy tắc khó bao quát những tình huống đa dạng.
- **Kỳ vọng vượt quá khả năng của công nghệ:** một số dự báo về tiến độ phát triển AI quá lạc quan.

Ví dụ, trong một bài toán tìm kiếm, nếu mỗi bước có nhiều lựa chọn thì tổng số khả năng có thể tăng theo cấp số nhân. Một phương pháp giải quyết được bài toán vài bước có thể trở nên quá chậm khi số bước tăng lên.

Giai đoạn này cho thấy việc xây dựng trí thông minh không chỉ đòi hỏi một ý tưởng thuật toán, mà còn cần dữ liệu, tài nguyên tính toán và cách đánh giá phù hợp.

1.5. Sự phát triển của hệ chuyên gia: thập niên 1970–1980

Trong khi mục tiêu tạo ra máy có trí thông minh tổng quát còn xa, các nhà nghiên cứu tập trung vào những hệ thống giải quyết tốt một lĩnh vực hẹp. Một hướng nổi bật là **hệ chuyên gia**, hay *Expert System*.

Hệ chuyên gia sử dụng tri thức do chuyên gia cung cấp để hỗ trợ suy luận và ra quyết định. Các thành phần thường gồm:

- **Cơ sở tri thức:** lưu các dữ kiện và quy tắc.
- **Bộ suy diễn:** áp dụng quy tắc để tìm kết luận.
- **Giao diện tương tác:** nhận thông tin và trình bày kết quả.

Quy tắc thường có dạng:

Nếu các điều kiện A và B được thỏa mãn thì đưa ra kết luận C.

Trong thập niên 1980, hệ chuyên gia tạo ra một làn sóng ứng dụng thương mại của AI, với những hệ thống tiêu biểu như XCON phục vụ cấu hình máy tính. Tuy nhiên, việc xây dựng và bảo trì các cơ sở quy tắc lớn trở nên tốn kém, góp phần dẫn đến đợt suy giảm tiếp theo vào cuối thập niên 1980 và đầu thập niên 1990. IBM

Hạn chế quan trọng của hệ chuyên gia là tri thức thường phải được đưa vào thủ công. Khi quy trình hoặc kiến thức chuyên ngành thay đổi, người phát triển phải cập nhật các quy tắc tương ứng.

Ngoài ra, những quy tắc mới có thể mâu thuẫn với quy tắc cũ. Hệ thống càng lớn thì việc kiểm tra tính nhất quán càng khó.

Dù gặp hạn chế, hệ chuyên gia đóng vai trò quan trọng trong lịch sử AI: nó chứng minh rằng một hệ thống chuyên biệt có thể tạo ra giá trị thực tế mà chưa cần đạt trí thông minh tổng quát.

1.6. Sự phát triển của học máy và mạng nơ-ron: thập niên 1980–2000

Một thay đổi quan trọng trong nghiên cứu AI là tăng cường sử dụng **học máy**, hay *Machine Learning*. Thay vì viết toàn bộ quy tắc, con người xây dựng mô hình để máy học các quan hệ từ dữ liệu.

Ví dụ, với bài toán nhận diện thư rác, người phát triển có thể cung cấp những email đã được gắn nhãn "thư rác" hoặc "không phải thư rác". Mô hình học các đặc điểm liên quan đến từng nhóm và sử dụng chúng để phân loại email mới.

Sự thay đổi này giúp AI giải quyết những bài toán khó mô tả đầy đủ bằng quy tắc thủ công.

Lan truyền ngược và mạng nơ-ron nhiều tầng

Năm 1986, David Rumelhart, Geoffrey Hinton và Ronald Williams công bố công trình có ảnh hưởng lớn về việc học các biểu diễn thông qua lan truyền ngược sai số. Công trình góp phần phổ biến phương pháp huấn luyện mạng nơ-ron nhiều tầng. Nature

Quá trình huấn luyện có thể hiểu qua bốn bước:

1. Đưa dữ liệu vào mạng để tính dự đoán.
2. So sánh dự đoán với kết quả đúng để tính sai số.
3. Tính ảnh hưởng của các trọng số đối với sai số.
4. Điều chỉnh trọng số nhằm giảm sai số.

Nhờ đó, các tầng ẩn có thể học những biểu diễn hữu ích từ dữ liệu. Tuy nhiên, huấn luyện các mạng lớn vẫn gặp khó khăn do tài nguyên tính toán và dữ liệu còn hạn chế.

LSTM và khả năng xử lý thông tin theo chuỗi

Năm 1997, Sepp Hochreiter và Jürgen Schmidhuber giới thiệu LSTM, một kiến trúc mạng nơ-ron hồi tiếp được thiết kế để cải thiện khả năng học các phụ thuộc dài trong chuỗi. MIT Press

Ý nghĩa của LSTM là giúp mô hình duy trì thông tin quan trọng qua nhiều bước. Điều này phù hợp với những bài toán mà dữ liệu hiện tại có liên quan đến thông tin xuất hiện trước đó, như văn bản, lời nói và chuỗi thời gian.

Deep Blue và thành công trong nhiệm vụ chuyên biệt

Cũng năm 1997, Deep Blue của IBM đánh bại nhà vô địch cờ vua thế giới Garry Kasparov trong một trận đấu sáu ván theo điều kiện thi đấu tiêu chuẩn. ibm.com

Deep Blue là minh chứng cho hiệu quả của việc kết hợp tìm kiếm, đánh giá thế cờ và năng lực tính toán lớn. Tuy nhiên, đây là một hệ thống chuyên chơi cờ vua; chiến thắng này không thể được diễn giải thành khả năng thông minh tổng quát trong mọi lĩnh vực.

1.7. Sự bùng nổ của học sâu: từ khoảng năm 2010

Học sâu, hay *Deep Learning*, là một nhánh của học máy sử dụng mạng nơ-ron có nhiều tầng để học các biểu diễn từ dữ liệu.

Ví dụ, trong nhận dạng ảnh, các tầng của mạng có thể học những thông tin ở nhiều mức:

- Tầng đầu học các đặc điểm đơn giản như cạnh và hướng.
- Các tầng tiếp theo kết hợp chúng thành những mẫu phức tạp hơn.
- Các tầng sâu sử dụng các biểu diễn đó để phục vụ nhận dạng hoặc phân loại.

Sự phát triển của học sâu được hỗ trợ bởi lượng dữ liệu số hóa lớn hơn, phần cứng tính toán mạnh hơn và các phương pháp huấn luyện hiệu quả hơn.

AlexNet năm 2012

Năm 2012, Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton công bố AlexNet, một mạng nơ-ron tích chập sâu đạt kết quả nổi bật trong cuộc thi nhận dạng ảnh ImageNet. Mô hình sử dụng GPU để huấn luyện, cùng các kỹ thuật như ReLU và dropout. proceedings.neurips.cc

Thành công này là một bước ngoặt của thị giác máy tính. Nó cho thấy mạng nơ-ron sâu có thể học những đặc trưng hiệu quả từ ảnh, thúc đẩy sự phát triển của các mô hình CNN trong những năm sau đó.

AlphaGo năm 2016

Tháng 3 năm 2016, AlphaGo của DeepMind đánh bại kỳ thủ Lee Sedol với tỷ số 4–1. Hệ thống kết hợp mạng nơ-ron sâu với các phương pháp tìm kiếm; học tăng cường cũng là một thành phần quan trọng trong quá trình phát triển của AlphaGo. Google DeepMind

Cờ vây có không gian nước đi rất lớn, khiến việc chỉ dựa vào tìm kiếm đơn thuần gặp khó khăn. AlphaGo sử dụng các mô hình học để hướng dẫn việc lựa chọn và đánh giá nước đi.

Sự kiện này thể hiện sức mạnh của việc kết hợp nhiều phương pháp AI trong một hệ thống.

1.8. Transformer và sự phát triển của mô hình ngôn ngữ lớn

Năm 2017, nhóm tác giả do Ashish Vaswani đứng đầu công bố bài báo *Attention Is All You Need*, giới thiệu kiến trúc Transformer. Kiến trúc được đề xuất sử dụng cơ chế attention thay cho cấu trúc hồi tiếp hoặc tích chập trong mô hình chuyển đổi chuỗi của bài báo. arxiv.org

Attention giúp mô hình tính toán mức độ liên quan giữa các vị trí trong chuỗi. Chẳng hạn, khi xử lý một câu, mô hình có thể sử dụng thông tin từ nhiều từ khác nhau để xây dựng biểu diễn cho một từ đang xét.

Một ưu điểm quan trọng của Transformer là khả năng xử lý song song nhiều vị trí trong quá trình huấn luyện. Điều này hỗ trợ việc huấn luyện các mô hình trên dữ liệu lớn.

Transformer trở thành nền tảng cho nhiều hệ thống xử lý ngôn ngữ sau đó. Sự phát triển này dẫn đến các mô hình ngôn ngữ lớn, được huấn luyện trên lượng văn bản lớn để học các mẫu ngôn ngữ và phục vụ nhiều nhiệm vụ.

Cần phân biệt: **Transformer là một kiến trúc mô hình**, còn **mô hình ngôn ngữ lớn là một nhóm mô hình theo quy mô và mục đích sử dụng**. Hai khái niệm có quan hệ chặt chẽ nhưng không đồng nghĩa.

1.9. AI tạo sinh và sự phổ biến của AI trong đời sống

AI tạo sinh, hay *Generative AI*, tập trung vào việc tạo ra nội dung như văn bản, hình ảnh, âm thanh hoặc mã chương trình dựa trên những mẫu và quan hệ đã học.

Khác với một hệ thống chỉ phân loại ảnh thành các nhóm, một hệ thống tạo sinh có thể tạo ra một ảnh mới từ mô tả. Trong xử lý ngôn ngữ, mô hình có thể sinh câu trả lời, tóm tắt nội dung hoặc tạo bản nháp văn bản.

Ngày 30 tháng 11 năm 2022, OpenAI giới thiệu ChatGPT dưới dạng bản nghiên cứu thử nghiệm. Giao diện hội thoại giúp người dùng tương tác với mô hình bằng ngôn ngữ tự nhiên và góp phần đưa AI tạo sinh đến gần công chúng hơn. OpenAI

Dấu mốc này thể hiện một thay đổi trong cách tiếp cận AI: người dùng có thể trực tiếp yêu cầu hệ thống thực hiện nhiệm vụ bằng lời nói hoặc văn bản, thay vì chỉ sử dụng AI như một chức năng hoạt động phía sau phần mềm.

Tuy nhiên, nội dung được tạo ra vẫn có thể sai, thiếu căn cứ hoặc phản ánh thiên lệch trong dữ liệu. Vì vậy, khả năng tạo câu trả lời trôi chảy cần được đánh giá cùng với độ chính xác và tính phù hợp.

1.10. Bảng tổng hợp các dấu mốc quan trọng

Thời điểm	**Dấu mốc**	**Ý nghĩa**
1943	McCulloch và Pitts xây dựng mô hình nơ-ron toán học	Đặt nền tảng cho nghiên cứu mạng nơ-ron nhân tạo
1950	Turing công bố *Computing Machinery and Intelligence*	Đưa ra cách nghiên cứu hành vi thông minh của máy
1955–1956	Đề xuất và chương trình nghiên cứu Dartmouth	Hình thành tên gọi và định hướng nghiên cứu AI
1958	Rosenblatt công bố công trình về Perceptron	Thúc đẩy nghiên cứu mô hình học từ dữ liệu
1965–1966	DENDRAL và ELIZA	Thể hiện khả năng suy luận chuyên ngành và hội thoại theo quy tắc
Thập niên 1970	Mùa đông AI thứ nhất	Bộc lộ khoảng cách giữa kỳ vọng và khả năng thực tế
Thập niên 1980	Hệ chuyên gia phát triển	Đưa AI vào những ứng dụng chuyên biệt
1986	Công trình về lan truyền ngược của Rumelhart, Hinton và Williams	Thúc đẩy huấn luyện mạng nơ-ron nhiều tầng
Cuối thập niên 1980–đầu 1990	Đợt suy giảm tiếp theo	Bộc lộ chi phí và hạn chế của các hệ thống AI đương thời
1997	LSTM và chiến thắng của Deep Blue	Tiến bộ trong xử lý chuỗi và giải quyết nhiệm vụ chuyên biệt
2012	AlexNet	Thúc đẩy sự bùng nổ của học sâu trong thị giác máy tính
2016	AlphaGo thắng Lee Sedol	Chứng minh hiệu quả của học sâu kết hợp tìm kiếm
2017	Transformer	Tạo nền tảng quan trọng cho nhiều mô hình ngôn ngữ
2022	ChatGPT được giới thiệu	Thúc đẩy sự phổ biến của AI tạo sinh qua giao diện hội thoại

Các giai đoạn trên có sự chồng lấn. AI dựa trên quy tắc vẫn được sử dụng sau khi học máy phát triển, và mạng nơ-ron đã được nghiên cứu từ lâu trước thời kỳ bùng nổ học sâu. Vì vậy, lịch sử AI nên được hiểu là quá trình nhiều hướng nghiên cứu cùng phát triển, bổ sung và kết hợp với nhau.

CHƯƠNG 2. CÁC KỸ THUẬT HỌC MÁY CƠ BẢN

Chương này trình bày các khái niệm nền tảng và bảy kỹ thuật học máy: hồi quy tuyến tính, hồi quy logistic, KNN, SVM tuyến tính, K-means, PCA và mạng nơ-ron truyền thẳng MLP. Mỗi kỹ thuật có công thức, giải thích nguyên lý và code tương ứng bằng **NumPy từ đầu — from scratch, Keras và PyTorch**.

Các đoạn code sử dụng những mảng số nhỏ để minh họa phép tính, không yêu cầu tải dataset. Đây là ví dụ phục vụ việc hiểu thuật toán, chưa phải thực nghiệm đánh giá hiệu quả trên dữ liệu thực tế.

2.1. Tổng quan về học máy

2.1.1. Khái niệm học máy

Học máy, hay *Machine Learning — ML*, là một lĩnh vực của trí tuệ nhân tạo nghiên cứu các phương pháp giúp máy tính học quy luật từ dữ liệu để đưa ra dự đoán hoặc quyết định.

Trong lập trình thông thường, người phát triển xây dựng các quy tắc xử lý. Trong học máy, người phát triển lựa chọn cấu trúc mô hình và phương pháp học; mô hình sử dụng dữ liệu để xác định những tham số hoặc quy luật phù hợp.

Có thể biểu diễn một mô hình dự đoán bằng công thức:

$$\hat{y} = f_\theta(x)$$

Trong đó:

- (x): đầu vào.
- $(\hat{y})$: đầu ra dự đoán.
- (f_θ): hàm biểu diễn mô hình.
- (θ): tập tham số của mô hình.

Ví dụ, với mô hình:

$$\hat{y} = wx + b$$

các tham số cần học là (w) và (b). Việc học nhằm tìm các giá trị tham số giúp dự đoán phù hợp với những ví dụ đã quan sát.

Không phải mọi thuật toán ML đều học một bộ trọng số. KNN chủ yếu lưu các mẫu tham chiếu và tìm những mẫu gần nhất khi dự đoán; PCA tìm các hướng biểu diễn dữ liệu; K-means tìm các tâm cụm.

2.1.2. Các nhóm phương pháp học máy

Nhóm phương pháp	**Đặc điểm**	**Nhiệm vụ thường gặp**
Học có giám sát	Sử dụng đầu vào và đầu ra đúng tương ứng	Hồi quy, phân loại

Nhóm phương pháp	Đặc điểm	Nhiệm vụ thường gặp
Học không giám sát	Không cần nhãn mục tiêu	Phân cụm, giảm chiều
Học bán giám sát	Kết hợp dữ liệu có nhãn và không có nhãn	Phân loại khi nhãn hạn chế
Học tăng cường	Học hành động thông qua tương tác và phần thưởng	Điều khiển, tìm chiến lược

Học có giám sát

Trong học có giám sát, mỗi ví dụ gồm một cặp:

$$[(x_i, y_i)]$$

Mô hình học quan hệ giữa (x_i) và (y_i), sau đó sử dụng quan hệ này để dự đoán cho đầu vào mới.

Hai nhiệm vụ chính là:

- **Hồi quy:** dự đoán giá trị liên tục.
- **Phân loại:** dự đoán một nhãn thuộc tập các lớp.

Học không giám sát

Trong học không giám sát, mô hình tìm cấu trúc trong các đầu vào \(x_i\) mà không sử dụng nhãn mục tiêu.

Hai nhiệm vụ tiêu biểu là:

- **Phân cụm:** nhóm các điểm tương tự nhau.
- **Giảm chiều:** tạo biểu diễn có ít đặc trưng hơn.

Học tăng cường

Trong học tăng cường, một tác nhân quan sát trạng thái, lựa chọn hành động và nhận phần thưởng từ môi trường. Mục tiêu là tìm chiến lược giúp tối đa hóa phần thưởng tích lũy kỳ vọng:

$$[J(\pi) = E_\pi \left[\sum_{t=0}^{T} \gamma^t r_t \right]]$$

Trong đó:

- (π): chính sách lựa chọn hành động.
- (r_t): phần thưởng tại bước \(t\).
- $(\gamma \in [0,1])$: hệ số chiết khấu.
- (T): thời điểm kết thúc.

Các phần tiếp theo tập trung vào những kỹ thuật cơ bản của học có giám sát và học không giám sát.

2.2. Các thành phần cơ bản của một mô hình ML

2.2.1. Đặc trưng và nhãn

Đặc trưng, hay *feature*, là những đại lượng dùng để biểu diễn một đối tượng.

Một đối tượng có \(d\) đặc trưng được biểu diễn bằng vector:

$$x = \begin{bmatrix} x_1 & x_2 & \cdots & x_d \end{bmatrix}^T$$

Với \(n\) đối tượng, ma trận đầu vào là:

$$X \in R^{n \times d}$$

Trong đó:

- Mỗi hàng biểu diễn một đối tượng.
- Mỗi cột biểu diễn một đặc trưng.

Nhãn (y) là kết quả cần dự đoán trong học có giám sát. Nhãn có thể là một giá trị liên tục hoặc một lớp phân loại.

2.2.2. Tham số và siêu tham số

Tham số được xác định trong quá trình học, chẳng hạn trọng số w, độ lệch b hoặc các tâm cụm.

Siêu tham số được lựa chọn để điều khiển quá trình học hoặc cấu trúc thuật toán, chẳng hạn:

- Learning rate.
- Số vòng lặp.
- Số láng giềng của KNN.
- Số cụm của K-means.
- Số nơ-ron trong tầng ẩn.

2.2.3. Hàm mất mát

Hàm mất mát đo mức độ khác biệt giữa dự đoán và kết quả đúng.

Với học có giám sát, mục tiêu thường có dạng:

$$\theta^* = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} l(f_\theta(x_i), y_i)]$$

Trong đó:

- (l): mất mát trên một ví dụ.
- (θ^*): bộ tham số cần tìm.

Các bài toán khác nhau sử dụng hàm mất mát khác nhau. Hồi quy thường sử dụng sai số bình phương; phân loại thường sử dụng cross-entropy hoặc hinge loss.

2.2.4. Gradient Descent

Gradient Descent cập nhật tham số theo hướng làm giảm hàm mục tiêu:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta \mathcal{L}}(\theta_t)$$

Trong đó:

- (θ_t): tham số tại bước (t).
- (η): learning rate.
- $(\nabla_{\theta \mathcal{L}})$: gradient của hàm mất mát.

Learning rate quá nhỏ khiến quá trình học chậm. Learning rate quá lớn có thể khiến quá trình cập nhật dao động hoặc không hội tụ.

Có ba cách tính gradient thường gặp:

Phương pháp	**Số ví dụ dùng cho một lần cập nhật**
Batch Gradient Descent	Toàn bộ các ví dụ
Stochastic Gradient Descent	Một ví dụ
Mini-batch Gradient Descent	Một nhóm ví dụ

Các ví dụ dưới đây chủ yếu dùng toàn bộ mảng minh họa trong mỗi lần cập nhật để dễ theo dõi công thức.

2.3. Chuẩn bị thư viện cho các đoạn code

Chạy đoạn này trước. Sau đó, trong mỗi mục, chạy mảng số minh họa rồi chọn phiên bản NumPy, Keras hoặc PyTorch cần sử dụng.

```python
import os

# Đặt trước khi import Keras.
os.environ.setdefault("KERAS_BACKEND", "tensorflow")

import numpy as np
import keras
from keras import layers, ops

import torch
from torch import nn

keras.utils.set_random_seed(42)
torch.manual_seed(42)

np.set_printoptions(precision=4, suppress=True)
```

Ba cách triển khai có vai trò như sau:

Cách triển khai	**Đặc điểm**
From scratch bằng NumPy	Tự viết các bước tính toán và cập nhật tham số
Keras	Dùng mô hình, tầng và cơ chế huấn luyện; với thuật toán cổ điển dùng keras.ops
PyTorch	Dùng tensor, các mô-đun và cơ chế tính gradient tự động

Trong Keras, tầng Dense thực hiện phép biến đổi tuyến tính kết hợp với hàm kích hoạt nếu được chỉ định. Keras cũng cung cấp API ops cho các phép toán tensor. keras.io

2.4. Hồi quy tuyến tính — Linear Regression

2.4.1. Khái niệm

Hồi quy tuyến tính mô hình hóa quan hệ tuyến tính giữa các đặc trưng đầu vào và một giá trị đầu ra liên tục.

Với một đặc trưng:

$$\hat{y} = wx + b$$

Với \(d\) đặc trưng:

$$\hat{y} = w^T x + b = \sum_{j=1}^{d} w_j x_j + b$$

Trong đó:

- (w_j): trọng số của đặc trưng thứ (j).

- (b): độ lệch.
- $(\hat{y})$: giá trị dự đoán.

2.4.2. Hàm mất mát

Sử dụng Mean Squared Error:

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Đặt:

$$[e_i = \hat{y}_i - y_i]$$

Gradient là:

$$[\frac{\partial \mathcal{L}}{\partial w} = \frac{2}{n}X^T e]$$

$$[\frac{\partial \mathcal{L}}{\partial b} = \frac{2}{n}\sum_{i=1}^{n} e_i]$$

Tham số được cập nhật bằng:

$$w \leftarrow w - \eta\frac{\partial \mathcal{L}}{\partial w}$$

$$b \leftarrow b - \eta\frac{\partial \mathcal{L}}{\partial b}$$

4.3. Mảng số minh họa

Các số dưới đây tuân theo quy luật \(y=2x+1\).

```
X_reg = np.array(
    [[0.0], [1.0], [2.0], [3.0]],
    dtype=np.float32
)

y_reg = np.array(
    [[1.0], [3.0], [5.0], [7.0]],
    dtype=np.float32
)

query_reg = np.array([[4.0]], dtype=np.float32)
```

a. From scratch bằng NumPy

```
w = np.zeros((1, 1))
b = np.zeros(1)

learning_rate = 0.05
n = len(X_reg)

for _ in range(2000):
    prediction = X_reg @ w + b
    error = prediction - y_reg

    dw = (2.0 / n) * X_reg.T @ error
    db = 2.0 * error.mean(axis=0)

    w -= learning_rate * dw
    b -= learning_rate * db

print("w:", w.ravel())
print("b:", b)
```

Giải thích:

- X_reg @ w + b: tính dự đoán.
- error: sai số của từng ví dụ.
- dw, db: gradient tự tính.
- Hai lệnh cập nhật cuối: thực hiện Gradient Descent.

Với mảng minh họa này, tham số học được gần w=2, b=1, nên dự đoán tại x=4 gần bằng 9.

b. Keras

```
linear_keras = keras.Sequential([
    keras.Input(shape=(1,)),
    layers.Dense(
        1,
        kernel_initializer="zeros",
        bias_initializer="zeros"
    )
])

linear_keras.compile(
    optimizer=keras.optimizers.SGD(learning_rate=0.05),
    loss="mse"
)

for _ in range(2000):
    linear_keras.train_on_batch(X_reg, y_reg)

w_keras, b_keras = linear_keras.layers[0].get_weights()
```

Một tầng Dense(1) không có hàm kích hoạt thực hiện đúng phép tính tuyến tính cần dùng trong mô hình này. Keras tự tính gradient và cập nhật tham số.

c. PyTorch

```
X_t = torch.tensor(X_reg)

y_t = torch.tensor(y_reg)

linear_torch = nn.Linear(1, 1)

with torch.no_grad():
    linear_torch.weight.zero_()
    linear_torch.bias.zero_()

criterion = nn.MSELoss()

optimizer = torch.optim.SGD(
    linear_torch.parameters(),
    lr=0.05
)

for _ in range(2000):
    optimizer.zero_grad()
    prediction = linear_torch(X_t)
    loss = criterion(prediction, y_t)
```

```
    loss.backward()
    optimizer.step()


with torch.no_grad():
    result = linear_torch(torch.tensor(query_reg))
print("w:", linear_torch.weight.detach().numpy())
print("b:", linear_torch.bias.detach().numpy())
print("Dự đoán:", result.numpy().ravel())
```

Trong PyTorch:

- zero_grad(): xóa gradient của lần cập nhật trước.
- backward(): tính gradient.
- step(): cập nhật tham số.

2.4.4. Ưu điểm và hạn chế

Ưu điểm của hồi quy tuyến tính là cấu trúc đơn giản, dễ giải thích và có chi phí tính toán tương đối thấp.

Hạn chế là mô hình chỉ biểu diễn trực tiếp quan hệ tuyến tính theo các đặc trưng được cung cấp. Mô hình cũng có thể bị ảnh hưởng mạnh bởi các điểm ngoại lệ khi sử dụng mất mát bình phương.

2.5. Hồi quy logistic — Logistic Regression

2.5.1. Khái niệm

Mặc dù có tên là "hồi quy", Logistic Regression thường được sử dụng cho bài toán phân loại.

Trong phân loại nhị phân, mô hình tính một giá trị tuyến tính:

$$z = w^T x + b$$

Sau đó sử dụng hàm sigmoid:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Giá trị p được dùng để biểu diễn xác suất dự đoán của lớp 1.

Với ngưỡng (0.5):

$$\hat{y} = \begin{cases} 1, & p \geq 0.5 \\ 0, & p < 0.5 \end{cases}$$

2.5.2. Hàm mất mát

Binary Cross-Entropy:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

Với sigmoid và mất mát này, gradient có dạng:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{n}X^T(p - y)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)$$

Logistic Regression một tầng có ranh giới quyết định tuyến tính trong không gian đặc trưng đầu vào.

2.5.3. Mảng số minh họa

```
X_cls = np.array(
    [[-2.0], [-1.0], [1.0], [2.0]],
    dtype=np.float32
)

y_cls = np.array(
    [[0.0], [0.0], [1.0], [1.0]],
    dtype=np.float32
)
```

a. From scratch bằng NumPy

```python
def sigmoid_numpy(z):
    z = np.clip(z, -50.0, 50.0)
    return 1.0 / (1.0 + np.exp(-z))


w = np.zeros((1, 1))
b = np.zeros(1)

learning_rate = 0.1
n = len(X_cls)

for _ in range(2000):
    logits = X_cls @ w + b
    probability = sigmoid_numpy(logits)

    error = probability - y_cls
    dw = X_cls.T @ error / n
    db = error.mean(axis=0)
    w -= learning_rate * dw
    b -= learning_rate * db

probability = sigmoid_numpy(X_cls @ w + b)
prediction = (probability >= 0.5).astype(int)

print("Xác suất:", probability.ravel())
print("Nhãn dự đoán:", prediction.ravel())
```

np.clip giới hạn đầu vào của hàm mũ nhằm tránh những giá trị quá lớn trong ví dụ.

b. Keras

```
logistic_keras = keras.Sequential([
    keras.Input(shape=(1,)),
    layers.Dense(
        1,
        activation="sigmoid",
        kernel_initializer="zeros",
        bias_initializer="zeros"
    )
])

logistic_keras.compile(
    optimizer=keras.optimizers.SGD(learning_rate=0.1),
    loss="binary_crossentropy"
)

for _ in range(2000):
    logistic_keras.train_on_batch(X_cls, y_cls)
```

Ở đây, mô hình xuất xác suất qua sigmoid nên sử dụng Binary Cross-Entropy với đầu vào là xác suất.

c. PyTorch

```
X_t = torch.tensor(X_cls)
y_t = torch.tensor(y_cls)

logistic_torch = nn.Linear(1, 1)

with torch.no_grad():
    logistic_torch.weight.zero_()
    logistic_torch.bias.zero_()

criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(
    logistic_torch.parameters(),
    lr=0.1
)

for _ in range(2000):
    optimizer.zero_grad()
```

```
    logits = logistic_torch(X_t)
    loss = criterion(logits, y_t)

    loss.backward()
    optimizer.step()

with torch.no_grad():
    probability = torch.sigmoid(logistic_torch(X_t))
    prediction = (probability >= 0.5).to(torch.int64)

print("Xác suất:", probability.numpy().ravel())
print("Nhãn dự đoán:", prediction.numpy().ravel())
```

BCEWithLogitsLoss nhận trực tiếp logits và kết hợp phép tính sigmoid với Binary Cross-Entropy theo cách ổn định số học. Vì vậy, không thêm sigmoid trước hàm mất mát này; sigmoid chỉ được dùng khi chuyển logits thành xác suất để xem kết quả. GitHub

2.5.4. Ưu điểm và hạn chế

Logistic Regression đơn giản, dễ triển khai và cung cấp xác suất dự đoán.

Hạn chế chính là ranh giới quyết định tuyến tính. Đối với bài toán phi tuyến, cần xây dựng thêm đặc trưng hoặc sử dụng một mô hình có khả năng biểu diễn phong phú hơn.

2.6. K láng giềng gần nhất — K-Nearest Neighbors

2.6.1. Khái niệm

KNN dự đoán dựa trên những mẫu gần nhất với đầu vào cần xét.

Với phân loại, thuật toán tìm (k) mẫu gần nhất rồi lựa chọn lớp xuất hiện nhiều nhất trong nhóm đó.

Với hồi quy, thuật toán có thể lấy trung bình giá trị mục tiêu của các mẫu gần nhất.

KNN không cần huấn luyện trọng số bằng Gradient Descent. Quá trình "học" chủ yếu là lưu các mẫu tham chiếu.

2.6.2. Công thức khoảng cách

Khoảng cách Euclid giữa hai vector:

$$d(x, x_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}$$

Gọi \(N_k(x)\) là tập \(k\) láng giềng gần nhất. Trong phân loại:

$$[\hat{y} = \arg\max_{c} \sum_{i \in N_k(x)} 1(y_i = c)]$$

Trong đó \(\mathbf{1}\) bằng \(1\) khi điều kiện đúng và bằng \(0\) khi điều kiện sai.

2.6.3. Mảng số minh họa

```
X_knn = np.array([
    [0.0, 0.0],
    [0.0, 1.0],
    [1.0, 0.0],
    [4.0, 4.0],
    [4.0, 5.0],
    [5.0, 4.0]
], dtype=np.float32)

y_knn = np.array([0, 0, 0, 1, 1, 1], dtype=np.int64)

query_knn = np.array([0.2, 0.3], dtype=np.float32)
k = 3
```

a. From scratch bằng NumPy

```
squared_distance = np.sum(
    (X_knn - query_knn) ** 2,
    axis=1
)

nearest_indices = np.argsort(squared_distance)[:k]
nearest_labels = y_knn[nearest_indices]

votes = np.bincount(nearest_labels, minlength=2)
prediction = np.argmax(votes)

print("Chỉ số láng giềng:", nearest_indices)
print("Nhãn láng giềng:", nearest_labels)
print("Nhãn dự đoán:", prediction)
```

Code sử dụng bình phương khoảng cách vì việc bỏ căn bậc hai không làm thay đổi thứ tự gần xa.

b. Keras

KNN được triển khai bằng keras.ops, không sử dụng tầng Dense hoặc lan truyền ngược.

```
X_k = ops.convert_to_tensor(X_knn)
y_k = ops.convert_to_tensor(y_knn, dtype="int32")
query_k = ops.convert_to_tensor(query_knn)

squared_distance = ops.sum(
    ops.square(X_k - query_k),
    axis=1
)

# top_k lấy các giá trị lớn nhất:
# đổi dấu để lấy những khoảng cách nhỏ nhất.
_, nearest_indices = ops.top_k(-squared_distance, k=k)

nearest_labels = ops.take(y_k, nearest_indices, axis=0)

votes = ops.sum(
    ops.one_hot(nearest_labels, num_classes=2),
    axis=0
```

ops.top_k trả về các giá trị lớn nhất và chỉ số tương ứng, nên sử dụng khoảng cách mang dấu âm để tìm các điểm gần nhất. keras.io

c. PyTorch

```
X_t = torch.tensor(X_knn)
y_t = torch.tensor(y_knn)
query_t = torch.tensor(query_knn)

squared_distance = ((X_t - query_t) ** 2).sum(dim=1)

nearest_indices = torch.topk(
    squared_distance,
    k=k,
    largest=False
).indices

nearest_labels = y_t[nearest_indices]
votes = torch.bincount(nearest_labels, minlength=2)
prediction = torch.argmax(votes)

print("Chỉ số láng giềng:", nearest_indices.numpy())
print("Nhãn dự đoán:", prediction.item())
```

2.6.4. Ưu điểm và hạn chế

KNN dễ hiểu và có thể tạo ranh giới phân loại phi tuyến thông qua cấu trúc phân bố của các mẫu.

Tuy nhiên:

- Dự đoán có thể chậm khi số mẫu tham chiếu lớn.
- Cần lưu các mẫu tham chiếu.
- Nhạy với thang đo đặc trưng.
- Kết quả phụ thuộc vào cách chọn \(k\) và khoảng cách.
- Hiệu quả có thể giảm trong không gian nhiều chiều.

Trong các đoạn code trên, nếu các lớp có số phiếu bằng nhau, argmax chọn lớp có chỉ số nhỏ hơn.

2.7. Máy vector hỗ trợ tuyến tính — Linear SVM

2.7.1. Khái niệm

SVM là một phương pháp phân loại tìm ranh giới quyết định kết hợp với việc kiểm soát biên phân tách giữa các lớp.

Với SVM tuyến tính, ranh giới là siêu phẳng:

$$w^T x + b = 0$$

Hàm chấm điểm:

$$f(x) = w^T x + b$$

Trong phân loại nhị phân với nhãn -1 và +1:

$$\hat{y} = \begin{cases} +1, & f(x) \geq 0 \\ -1, & f(x) < 0 \end{cases}$$

2.7.2. Biên phân tách và hàm mục tiêu

Với cách chuẩn hóa hai mặt biên:

$$[w^T x + b = +1]$$

$$[w^T x + b = -1]$$

khoảng cách giữa chúng là:

$$\frac{2}{|w|_2}$$

Giảm độ lớn của w giúp tăng độ rộng biên, nhưng cần đồng thời kiểm soát các vi phạm phân loại.

Một hàm mục tiêu dạng soft-margin là:

$$\mathcal{L} = \frac{\lambda}{2}\|w\|_2^2 + \frac{1}{n}\sum_{i=1}^{n} \max\left(0, 1 - y_i(w^T x_i + b)\right)$$

Trong đó:

- Thành phần thứ nhất là regularization.
- Thành phần thứ hai là hinge loss.
- $(\lambda > 0)$ điều khiển mức độ regularization.

Hinge loss bằng 0 khi điểm có nhãn đúng và đạt biên yêu cầu. Điểm có $(y_i f(x_i) < 1)$ vẫn bị phạt, kể cả khi dự đoán đúng nhưng nằm quá gần ranh giới. SVM còn có các biến thể dùng kernel; phần code này minh họa SVM tuyến tính trong dạng tối ưu nguyên thủy.

2.7.3. Mảng số minh họa

```
X_svm = np.array(
    [[-2.0], [-1.0], [1.0], [2.0]],
    dtype=np.float32
)

y_svm = np.array(
    [[-1.0], [-1.0], [1.0], [1.0]],
    dtype=np.float32
)

lambda_svm = 0.01
```

a. From scratch bằng NumPy

Hinge loss không khả vi tại điểm gãy; code sử dụng một lựa chọn subgradient.

```
w = np.zeros((1, 1))
b = np.zeros(1)

learning_rate = 0.05
n = len(X_svm)

for _ in range(1000):
    score = X_svm @ w + b
    margin = y_svm * score

    active = (margin < 1.0).astype(float)

    dw = (
        lambda_svm * w
        - X_svm.T @ (active * y_svm) / n
    )
```

```
    db = -(active * y_svm).mean(axis=0)

    w -= learning_rate * dw
    b -= learning_rate * db

score = X_svm @ w + b
prediction = np.where(score >= 0.0, 1, -1)

print("Điểm số:", score.ravel())
print("Nhãn dự đoán:", prediction.ravel())
```

active đánh dấu các điểm chưa đạt điều kiện biên. Chỉ các điểm này đóng góp vào phần subgradient của hinge loss.

b. Keras

```
svm_keras = keras.Sequential([
    keras.Input(shape=(1,)),
    layers.Dense(
        1,
        kernel_initializer="zeros",
        bias_initializer="zeros",
        kernel_regularizer=keras.regularizers.L2(
            lambda_svm / 2.0
        )
    )
])

svm_keras.compile(
    optimizer=keras.optimizers.SGD(learning_rate=0.05),
    loss=keras.losses.Hinge()
)
```

```
for _ in range(1000):
    svm_keras.train_on_batch(X_svm, y_svm)

score = svm_keras.predict(X_svm, verbose=0)
prediction = np.where(score >= 0.0, 1, -1)

print("Điểm số:", score.ravel())
print("Nhãn dự đoán:", prediction.ravel())
```

Mô hình xuất điểm số tuyến tính, không sử dụng sigmoid. Regularization L2 trên trọng số bổ sung thành phần \(\lambda\|w\|^2/2\) vào hàm mục tiêu.

c. PyTorch

```
X_t = torch.tensor(X_svm)
y_t = torch.tensor(y_svm)

svm_torch = nn.Linear(1, 1)

with torch.no_grad():
    svm_torch.weight.zero_()
    svm_torch.bias.zero_()

optimizer = torch.optim.SGD(
    svm_torch.parameters(),
    lr=0.05
)

for _ in range(1000):
    optimizer.zero_grad()
```

2.7.4. Ưu điểm và hạn chế

SVM tuyến tính có mục tiêu tối ưu rõ ràng và phù hợp với nhiều bài toán có số đặc trưng lớn.

Hạn chế là ranh giới vẫn tuyến tính. Hiệu quả phụ thuộc vào biểu diễn đặc trưng và mức regularization. Điểm số của SVM cũng không tự động là xác suất dự đoán.

CHƯƠNG 3.CNN

1.Tổng quan kiến thức CNN

- **Khái niệm:** CNN (Convolutional Neural Network) là mạng nơ-ron tích chập, được sử dụng phổ biến để xử lý và nhận dạng hình ảnh. CNN có khả năng tự học đặc trưng từ dữ liệu, từ các đặc trưng đơn giản như cạnh, đường nét đến những đặc trưng phức tạp như hình dạng và cấu trúc đối tượng.
- **Các thành phần chính:** Lớp **Convolution** sử dụng các bộ lọc (*kernel*) trượt trên ảnh để trích xuất đặc trưng. Hàm **ReLU** tạo tính phi tuyến, giúp mô hình học các quan hệ phức tạp. Lớp **Pooling** giảm kích thước bản đồ đặc trưng và lượng tính toán. Cuối cùng, **Flatten hoặc Global Average Pooling** chuyển đặc trưng thành vector để lớp phân loại đưa ra dự đoán.
- **Nguyên lý hoạt động:** Ảnh đầu vào được xử lý qua nhiều lớp liên tiếp. Các lớp đầu thường phát hiện đường nét và họa tiết; các lớp sâu hơn kết hợp chúng để nhận diện những cấu trúc phức tạp. Trong quá trình huấn luyện, mô hình tính sai lệch giữa dự đoán và nhãn thật bằng hàm mất mát, sau đó lan truyền ngược để điều chỉnh trọng số.
- **Ưu điểm:** CNN tự động trích xuất đặc trưng, giảm nhu cầu thiết kế đặc trưng thủ công. Nhờ **kết nối cục bộ và chia sẻ trọng số**, mô hình khai thác tốt cấu trúc không gian của ảnh và giảm số lượng tham số so với mạng kết nối đầy đủ xử lý trực tiếp ảnh.
- **Hạn chế:** Huấn luyện CNN từ đầu thường cần nhiều dữ liệu và tài nguyên tính toán. Mô hình có thể bị quá khớp khi dữ liệu ít, hoặc dự đoán kém khi ảnh thực tế khác

nhiều so với ảnh huấn luyện. Các kỹ thuật như tăng cường dữ liệu, Dropout và học chuyển giao thường được dùng để cải thiện khả năng tổng quát hóa.

- **Ứng dụng và kiến trúc tiêu biểu:** CNN được ứng dụng trong phân loại ảnh, nhận dạng chữ viết, phát hiện đối tượng, phân đoạn ảnh và kiểm tra chất lượng sản phẩm. Một số kiến trúc phổ biến gồm **LeNet, VGG, ResNet, MobileNet và EfficientNet**, với những cải tiến nhằm nâng cao độ chính xác hoặc giảm chi phí tính toán.

2.Giới thiệu và dataset

a. Fashion MNIST

Link: https://www.kaggle.com/datasets/zalando-research/fashionmnist/data

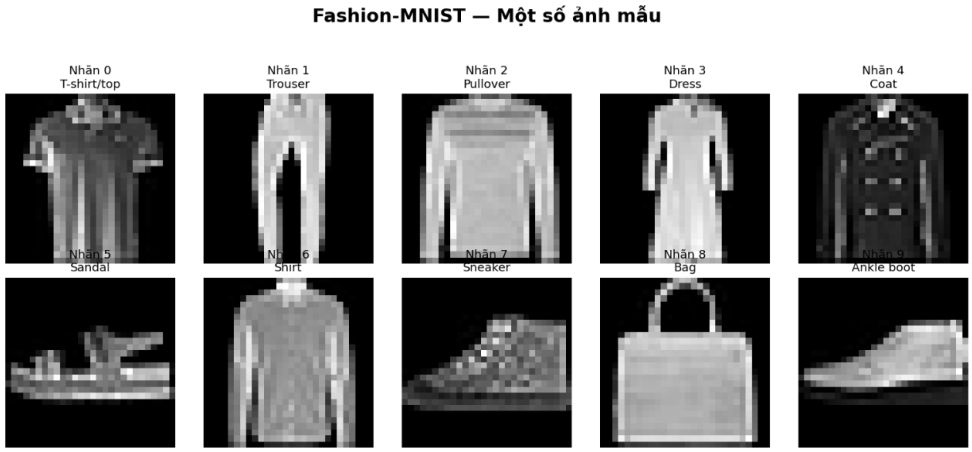
1. Tổng quan kích cỡ và cấu trúc bộ dữ liệu Fashion-MNIST



Nhận xét:

- **Quy mô dữ liệu:** Fashion-MNIST gồm **70.000 ảnh**, cung cấp nhiều mẫu để thực nghiệm bài toán phân loại sản phẩm thời trang. Quy mô này phù hợp để huấn luyện và so sánh các mô hình CNN trong phạm vi bài thực hành.
- **Kích thước ảnh:** Mỗi ảnh có kích thước **28 × 28 pixel**, tương ứng **784 pixel**. Kích thước nhỏ giúp giảm lượng dữ liệu cần xử lý và chi phí tính toán khi huấn luyện. Tuy nhiên, một số chi tiết nhỏ như họa tiết, đường may hoặc cổ áo có thể khó quan sát.
- **Số kênh ảnh:** Các ảnh đều là **ảnh xám một kênh**, vì vậy thông tin đầu vào chủ yếu thể hiện mức sáng và hình dạng sản phẩm. Mô hình cần khai thác đường viền, cấu trúc và sự thay đổi mức xám để nhận diện đối tượng.
- **Số lớp:** Dataset gồm **10 lớp sản phẩm**, tạo thành bài toán phân loại đa lớp. Đầu ra của mô hình cần thể hiện khả năng ảnh thuộc từng lớp và xác định nhãn dự đoán tương ứng.
- **Tính thống nhất:** Các ảnh có cùng kích thước và số kênh, thuận tiện cho việc tạo các batch và đưa vào CNN. Dữ liệu không cần thay đổi kích thước ảnh để thống nhất đầu vào.

2. Minh họa ảnh mẫu của 10 lớp trong Fashion-MNIST



Nhận xét:

- **Nội dung ảnh mẫu:** Hình hiển thị một ảnh đại diện cho mỗi lớp, gồm áo thun, quần dài, áo chui đầu, váy, áo khoác, sandal, áo sơ mi, giày thể thao, túi và bốt cổ thấp. Nhãn từ **0 đến 9** được ghi cùng tên lớp, giúp xác định mối liên hệ giữa ảnh đầu vào và nhãn phân loại.
- **Đặc điểm nền ảnh:** Trong các mẫu được hiển thị, sản phẩm nằm trên nền đen và được thể hiện bằng các mức xám khác nhau. Nền tương đối đơn giản giúp quan sát hình dạng và đường viền sản phẩm rõ hơn.
- **Sự khác biệt giữa các lớp:** Quần dài có cấu trúc hai ống quần; túi có phần thân và quai; sandal có những khoảng trống giữa các bộ phận. Những đặc điểm này tạo ra các dấu hiệu hình dạng có thể hỗ trợ việc phân loại.
- **Sự tương đồng giữa các lớp:** Các nhóm áo thun, áo chui đầu, áo khoác và áo sơ mi đều có phần thân và tay áo, nên có thể xuất hiện những đặc trưng giống nhau. Mô hình cần nhận biết thêm các chi tiết như độ dài tay áo, cổ áo, phần mở phía trước và đường viền để phân biệt.
- **Chất lượng biểu diễn:** Do độ phân giải thấp, các ảnh xuất hiện dạng ô pixel và một số chi tiết bị giản lược. Điều này có thể làm những sản phẩm có hình dáng gần nhau khó phân biệt hơn.
- **Ý nghĩa đối với CNN:** Các ảnh mẫu cho thấy bài toán cần khai thác cả đặc trưng cục bộ, như cạnh và họa tiết, lẫn hình dạng tổng thể. Đây là cơ sở để sử dụng các tầng tích chập nhằm học đặc trưng phục vụ phân loại. Khả năng phân biệt thực tế giữa các lớp sẽ được kiểm chứng sau khi huấn luyện.

3. Phân bố số lượng ảnh theo lớp trong Fashion-MNIST



Nhận xét:

- **Số lượng mẫu từng lớp:** Mỗi lớp có **7.000 ảnh**, chiếm **10%** tổng số 70.000 ảnh. Không có lớp nào có số lượng mẫu nhiều hoặc ít hơn các lớp còn lại.
- **Đặc điểm biểu đồ:** Cả 10 cột đều có chiều cao bằng nhau. Các giá trị được ghi trên đầu cột xác nhận dataset cân bằng hoàn toàn về số lượng nhãn.
- **Ý nghĩa đối với huấn luyện:** Phân bố cân bằng giúp hạn chế việc mô hình thiên về một lớp chỉ vì lớp đó xuất hiện thường xuyên hơn trong dữ liệu. Xét về số lượng, các lớp đều cung cấp lượng mẫu ngang nhau để mô hình học đặc trưng.
- **Ý nghĩa đối với đánh giá:** Khi các lớp có số lượng ngang nhau, kết quả tổng thể ít bị chi phối bởi một lớp đông mẫu. Tuy vậy, vẫn cần xem các chỉ số theo từng lớp để phát hiện những nhóm sản phẩm mà mô hình nhận diện chưa tốt.
- **Giới hạn của phân bố nhãn:** Số lượng bằng nhau không đồng nghĩa độ đa dạng hoặc độ khó của các lớp giống nhau. Một lớp có nhiều kiểu dáng khác nhau, hoặc có hình dạng tương tự lớp khác, vẫn có thể khó phân loại hơn.
- **Kết luận từ biểu đồ:** Fashion-MNIST không có tình trạng mất cân bằng số lượng lớp trên toàn bộ dataset. Vì vậy, trọng tâm thực nghiệm có thể tập trung vào khả năng học đặc trưng và phân biệt sản phẩm của các mô hình CNN.

b. Kuzushiji-MNIST

Link: https://www.kaggle.com/datasets/anokas/kuzushiji/data

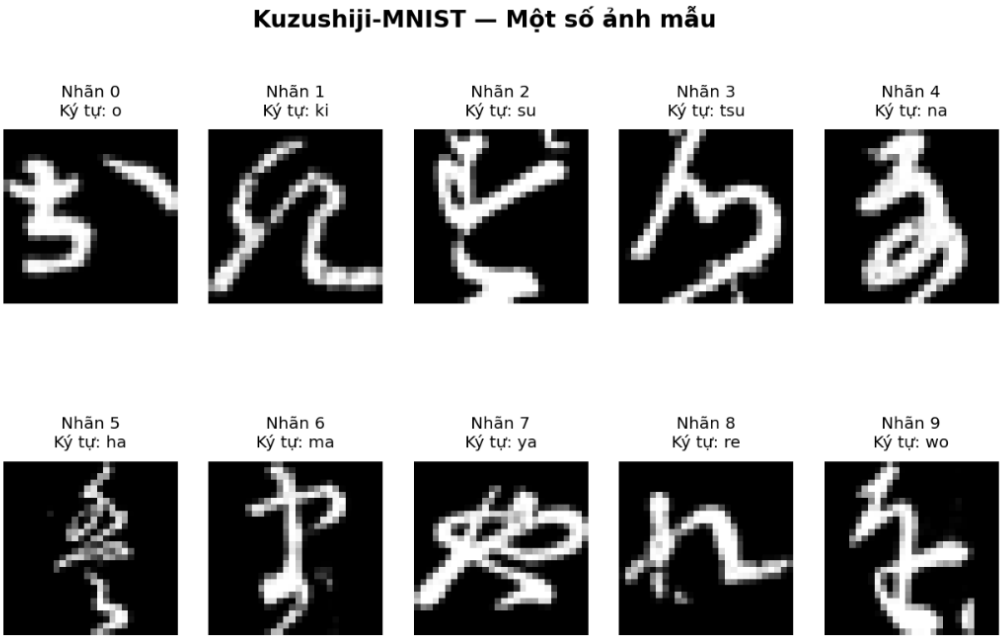
1. Tổng quan kích cỡ và cấu trúc bộ dữ liệu Kuzushiji-MNIST



Nhận xét:

- **Quy mô dữ liệu:** Kuzushiji-MNIST gồm **70.000 ảnh ký tự**, cung cấp nhiều mẫu cho bài toán nhận diện chữ viết cổ của Nhật Bản. Quy mô này phù hợp để thực nghiệm và so sánh khả năng phân loại của các mô hình CNN.
- **Kích thước ảnh:** Mỗi ảnh có kích thước **28 × 28 pixel**, tương ứng **784 pixel**. Kích thước nhỏ giúp giảm lượng tính toán, nhưng cũng hạn chế khả năng biểu diễn những nét chữ nhỏ hoặc nằm gần nhau.
- **Số kênh ảnh:** Dữ liệu sử dụng **ảnh xám một kênh**. Thông tin nhận diện chủ yếu nằm ở hình dạng nét chữ, đường cong, vị trí các nét và sự thay đổi mức xám.
- **Số lớp:** Bộ dữ liệu gồm **10 lớp ký tự**, được mã hóa bằng nhãn từ **0 đến 9**. Đây là bài toán phân loại đa lớp, trong đó mô hình cần xác định ảnh đầu vào thuộc loại ký tự nào.
- **Tính thống nhất:** Các ảnh có cùng kích thước và số kênh, thuận tiện cho việc tiền xử lý và đưa vào CNN. Cấu trúc đầu vào tương đồng với Fashion-MNIST, giúp thuận tiện khi áp dụng cùng một kiến trúc để thực nghiệm trên hai dataset.

2. Minh họa ảnh mẫu của 10 lớp trong Kuzushiji-MNIST

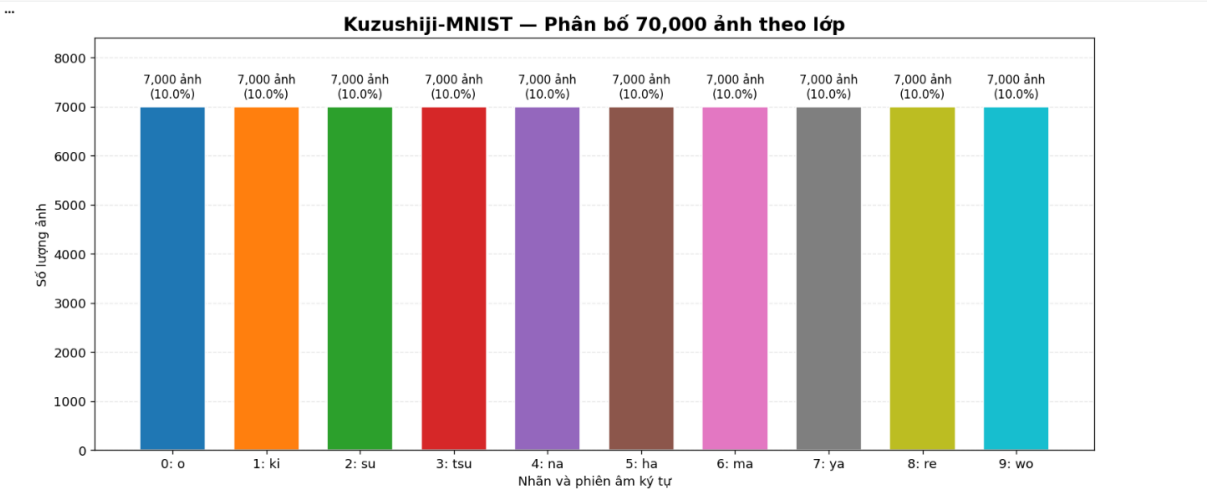


Nhận xét:

- **Nội dung ảnh mẫu:** Hình hiển thị một ảnh đại diện cho mỗi lớp. Nhãn từ **0 đến 9** tương ứng với các ký tự có phiên âm **o, ki, su, tsu, na, ha, ma, ya, re và wo**. Các tên phiên âm giúp xác định lớp của ảnh; hình ảnh thể hiện dạng chữ viết cổ.
- **Đặc điểm nền ảnh:** Trong các mẫu được hiển thị, nét chữ sáng nằm trên nền đen, tạo độ tương phản tương đối rõ. Điều này giúp quan sát đường nét và hình dạng của từng ký tự.

- **Đặc điểm nét chữ:** Các mẫu có nhiều đường cong, nét nối và điểm giao nhau. Độ dày, hướng nét và diện tích nét chữ khác nhau giữa các ảnh, tạo ra những đặc trưng mà mô hình có thể khai thác.
- **Sự tương đồng về hình dạng:** Một số mẫu cùng xuất hiện nét cong hoặc vòng khép, nên chỉ dựa vào một đoạn nét riêng lẻ có thể chưa đủ để nhận diện. Mô hình cần kết hợp đặc trưng cục bộ với vị trí và cấu trúc tổng thể của ký tự.
- **Ảnh hưởng của độ phân giải:** Khi phóng lớn, các ảnh xuất hiện dạng ô pixel. Một số nét nhỏ hoặc gần nhau khó phân biệt rõ, có thể làm tăng độ khó của bài toán nhận diện.
- **Ý nghĩa đối với CNN:** Các tầng tích chập có thể học những đặc trưng như cạnh, đường cong và điểm giao nét, sau đó kết hợp chúng để phân loại ký tự. Tuy nhiên, mỗi lớp mới được minh họa bằng một ảnh, nên hình này chưa thể hiện đầy đủ sự đa dạng cách viết trong cùng lớp.

3. Phân bố số lượng ảnh theo lớp trong Kuzushiji-MNIST



Nhận xét:

- **Số lượng mẫu từng lớp:** Mỗi lớp có **7.000 ảnh**, chiếm **10%** tổng số 70.000 ảnh. Không có lớp nào chiếm ưu thế hoặc có số lượng mẫu thấp hơn các lớp còn lại.
- **Đặc điểm biểu đồ:** Cả 10 cột đều có chiều cao bằng nhau. Giá trị số lượng và tỷ lệ ghi trên đầu cột xác nhận bộ dữ liệu cân bằng hoàn toàn về số lượng nhãn.
- **Ý nghĩa của sự cân bằng:** Các lớp có lượng mẫu ngang nhau, giúp hạn chế ảnh hưởng của chênh lệch số lượng lớp trong thực nghiệm. Đây là điều kiện thuận lợi để nghiên cứu khả năng phân biệt các dạng ký tự của mô hình.
- **Giới hạn của phân bố nhãn:** Số lượng bằng nhau không đồng nghĩa độ đa dạng cách viết hoặc độ khó nhận diện của các lớp giống nhau. Những ký tự có cấu trúc nét tương tự vẫn có thể gây nhầm lẫn.
- **Ý nghĩa đối với đánh giá:** Ngoài độ chính xác tổng thể, cần xem Precision, Recall, F1-score theo lớp và ma trận nhầm lẫn để xác định các nhóm ký tự mô hình nhận diện tốt hoặc còn hạn chế.

- **Liên hệ với Fashion-MNIST:** Hai dataset đều có **70.000 ảnh, 10 lớp và 7.000 ảnh mỗi lớp**. Sự tương đồng về quy mô giúp thuận tiện cho việc bố trí thực nghiệm, còn nội dung ảnh khác nhau cho phép đánh giá CNN trên cả sản phẩm thời trang và ký tự viết tay.

3. Các mô hình CNN

3.1. CNN cơ bản – Mạng tích chập nền tảng cho phân loại hình ảnh

- **Khái niệm:** CNN cơ bản là mạng nơ-ron sử dụng phép tích chập để học đặc trưng từ hình ảnh, sau đó kết hợp các đặc trưng nhằm xác định lớp của đối tượng.
- **Cấu trúc:** Trong bài này, mô hình gồm **hai lớp tích chập**, kết hợp ReLU và Max Pooling, tiếp theo là Flatten và các lớp kết nối đầy đủ để phân loại.
- **Trích xuất đặc trưng:** Các bộ lọc trượt trên ảnh và tạo bản đồ đặc trưng. Lớp đầu học những đặc trưng đơn giản như cạnh và đường nét; lớp tiếp theo kết hợp chúng thành các đặc trưng phức tạp hơn.
- **Xử lý trung gian:** ReLU tạo tính phi tuyến, còn Max Pooling giảm kích thước không gian của bản đồ đặc trưng. Flatten chuyển các bản đồ này thành một vector.
- **Đầu ra:** Lớp phân loại tạo điểm số cho từng lớp. Với FashionMNIST và Kuzushiji-MNIST, đầu ra gồm **10 điểm số**, có thể chuyển thành xác suất bằng Softmax.

3.2. VGG16 – Mạng tích chập sâu với các bộ lọc nhỏ

- **Khái niệm:** VGG16 là kiến trúc CNN được giới thiệu bởi nhóm Visual Geometry Group của Đại học Oxford, sử dụng nhiều lớp tích chập xếp chồng để học đặc trưng hình ảnh.
- **Cấu trúc:** Kiến trúc chuẩn gồm **13 lớp tích chập và 3 lớp kết nối đầy đủ**, tạo thành 16 lớp có trọng số. Phần trích xuất đặc trưng được chia thành **5 khối tích chập**.
- **Bộ lọc:** Các lớp tích chập sử dụng kernel (3×3), stride 1 và padding 1. Nhiều lớp liên tiếp giúp mở rộng vùng tiếp nhận mà vẫn giữ kích thước không gian trong mỗi khối.
- **Giảm kích thước:** Sau mỗi khối, Max Pooling (2×2), stride 2 giảm chiều cao và chiều rộng. Số kênh tăng qua các khối theo thứ tự **64, 128, 256, 512 và 512**.
- **Phân loại:** Đặc trưng cuối được trải phẳng và đưa qua các lớp kết nối đầy đủ. Khi áp dụng vào hai dataset đã chọn, lớp đầu ra được điều chỉnh thành **10 lớp**.

3.3. ResNet50 – Mạng tích chập sâu sử dụng học phần dư

- **Khái niệm:** ResNet50 là kiến trúc CNN thuộc họ Residual Network, được thiết kế theo nguyên lý **học phần dư** và có 50 lớp theo quy ước đếm của kiến trúc.
- **Cấu trúc:** Mạng gồm lớp tích chập đầu vào, các nhóm khối residual, Global Average Pooling và lớp kết nối đầy đủ. Bốn nhóm residual lần lượt có **3, 4, 6 và 3 khối bottleneck**.

- **Khối bottleneck:** Mỗi khối gồm ba lớp tích chập (1×1), (3×3) và (1×1), lần lượt biến đổi số kênh, xử lý đặc trưng không gian và đưa số kênh về kích thước đầu ra của khối.
- **Kết nối tắt:** Đầu vào được truyền qua một nhánh tắt và cộng với kết quả của nhánh tích chập: $(y = F(x) + x)$. Khi kích thước khác nhau, nhánh tắt sử dụng phép biến đổi để các tensor có thể cộng được.
- **Phân loại:** Global Average Pooling lấy trung bình trên từng kênh để tạo vector đặc trưng. Vector này được đưa vào lớp kết nối đầy đủ, với đầu ra điều chỉnh thành **10 lớp** cho bài toán đang xét.

4.Thực nghiệm trên các tập dataset

4.1. Dataset 1

4.1.1. Thiết lập môi trường và cấu hình GPU cho huấn luyện CNN Scratch

```
%pip install -q pandas matplotlib scikit-learn tqdm

import sys
import re
import subprocess
import importlib.util
import io
import zipfile
import json
import time
import gc
from pathlib import Path

# Kiểm tra GPU của Colab
gpu_info = subprocess.run(
    "nvidia-smi", shell=True, capture_output=True, text=True
)
if gpu_info.returncode != 0:
    raise RuntimeError(
        "Hãy bật GPU: Runtime → Change runtime type → GPU."
    )

# Giữ CuPy đang có; nếu chưa có thì cài theo phiên bản CUDA.
if importlib.util.find_spec("cupy") is None:
    match = re.search(r"CUDA Version:\s*(\d+)", gpu_info.stdout)
    cuda_major = int(match.group(1)) if match else 12

    if cuda_major < 12:
        raise RuntimeError("Notebook này cần GPU có CUDA 12 trở lên.")

    package = "cupy-cuda13x" if cuda_major >= 13 else "cupy-cuda12x"
    subprocess.check_call([
        sys.executable, "-m", "pip", "install", "-q",
        f"{package}[ctk]"
    ])

import cupy as cp
import numpy as np
import pandas as pdA
import matplotlib.pyplot as plt

from google.colab import files
from tqdm.auto import tqdm
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    precision_recall_fscore_support,
    confusion_matrix
```

- **Cài đặt và nhập các thư viện cần thiết:** Chuẩn bị Pandas để xử lý dữ liệu, Matplotlib để vẽ biểu đồ, Scikit-learn để chia dữ liệu và tính các chỉ số đánh giá. Thư viện Tqdm giúp hiển thị tiến độ huấn luyện.

- **Kiểm tra GPU và thiết lập CuPy:** Sử dụng nvidia-smi để kiểm tra GPU trên Colab. Nếu chưa có CuPy, chương trình chọn gói cài đặt theo thông tin CUDA; sau đó kiểm tra GPU khả dụng và in tên GPU, phiên bản CuPy.
- **Cấu hình tính toán và tính tái lập:** Gán xp = cp để các phép tính Scratch ở những cell tiếp theo sử dụng CuPy trên GPU. Đặt SEED = 42 cho NumPy và CuPy nhằm tăng khả năng tái lập kết quả khi chạy lại.
- **Khai báo nhãn và mô hình:** Xác định danh sách 10 lớp trang phục của FashionMNIST và tên ba mô hình gồm BasicCNN, VGG16, ResNet50. Các danh sách này giúp thống nhất tên lớp và tên mô hình trong quá trình huấn luyện, đánh giá và trực quan hóa.

4.1.2. Tải và tiền xử lý dữ liệu FashionMNIST từ Kaggle

```
dataset_path = kagglehub.dataset_download(
    "zalando-research/fashionmnist"
)
data_dir = Path(dataset_path)

print("Đã tải dataset tại:", data_dir)

# 2. Tìm hai file CSV trong dataset
def find_file(filename):
    matches = list(data_dir.rglob(filename))
    if len(matches) != 1:
        raise FileNotFoundError(
            f"Cần đúng một file {filename}, tìm thấy {len(matches)}."
        )
    return matches[0]

train_file = find_file("fashion-mnist_train.csv")
test_file = find_file("fashion-mnist_test.csv")

df_train = pd.read_csv(train_file, dtype=np.uint8)
df_test = pd.read_csv(test_file, dtype=np.uint8)

assert df_train.shape == (60000, 785), "Sai kích thước CSV train."
assert df_test.shape == (10000, 785), "Sai kích thước CSV test."
assert "label" in df_train.columns and "label" in df_test.columns

# 3. Tách nhãn và ảnh 28×28
y_full = df_train["label"].to_numpy(dtype=np.int64)
y_test = df_test["label"].to_numpy(dtype=np.int64)

images_full = df_train.drop(columns="label").to_numpy(
    dtype=np.uint8
).reshape(-1, 28, 28)

images_test = df_test.drop(columns="label").to_numpy(
    dtype=np.uint8
).reshape(-1, 28, 28)

assert np.isin(y_full, np.arange(10)).all()
assert np.isin(y_test, np.arange(10)).all()

# 4. Chia 60.000 ảnh thành 54.000 train và 6.000 validation
# Giữ nguyên 10.000 ảnh test của dataset.
train_ids, val_ids = train_test_split(
    np.arange(len(y_full)),
    test_size=0.10,
    random_state=SEED,
    stratify=y_full
)
```

```python
# 5. Thêm viền đen để chuyển 28×28 thành 32×32
# Chuẩn hóa về [-1, 1] và chuyển thành dạng NCHW.
def prepare_images(images):
    padded = np.pad(
        images,
        ((0, 0), (2, 2), (2, 2)),
        mode="constant",
        constant_values=0
    )
    return (
        padded[:, None].astype(np.float32) / 127.5 - 1.0
    )

X_train = prepare_images(images_full[train_ids])
X_val = prepare_images(images_full[val_ids])
X_test = prepare_images(images_test)

y_train = y_full[train_ids]
y_val = y_full[val_ids]

print("\nDữ liệu đã sẵn sàng:")
print("Train:", X_train.shape, "| Nhãn:", y_train.shape)
print("Validation:", X_val.shape, "| Nhãn:", y_val.shape)
print("Test:", X_test.shape, "| Nhãn:", y_test.shape)
print("Khoảng giá trị pixel:", X_train.min(), "đến", X_train.max())

# 6. Giải phóng dữ liệu trung gian
del df_train, df_test, images_full, images_test
del y_full, train_ids, val_ids
gc.collect()
```

```
Using Colab cache for faster access to the 'fashionmnist' dataset.
Đã tải dataset tại: /kaggle/input/fashionmnist

Dữ liệu đã sẵn sàng:
Train: (54000, 1, 32, 32) | Nhãn: (54000,)
Validation: (6000, 1, 32, 32) | Nhãn: (6000,)
Test: (10000, 1, 32, 32) | Nhãn: (10000,)
Khoảng giá trị pixel: -1.0 đến 1.0
0
```

- Tải và kiểm tra dữ liệu: Sử dụng kagglehub để tải FashionMNIST trực tiếp vào Colab, sau đó tìm và đọc hai file CSV train, test. Kiểm tra dữ liệu có đúng 60.000 ảnh train, 10.000 ảnh test và 785 cột gồm một cột nhãn, 784 cột pixel.
- Tách ảnh và nhãn: Lấy cột label làm nhãn, chuyển 784 giá trị pixel của mỗi mẫu thành ảnh xám 28×28. Kiểm tra các nhãn đều thuộc khoảng 0–9, tương ứng với 10 loại trang phục.
- Chia tập huấn luyện và validation: Chia 60.000 ảnh thành 54.000 ảnh train và 6.000 ảnh validation, giữ nguyên 10.000 ảnh test. Tham số stratify giúp duy trì tỷ lệ các lớp giữa train và validation; random_state=42 giúp tái lập cách chia.
- Chuẩn hóa và định dạng đầu vào: Thêm viền đen 2 pixel mỗi phía để chuyển ảnh thành 32×32, chuẩn hóa pixel về [-1, 1] và chuyển sang float32. Dữ liệu có dạng (N, 1, 32, 32) phù hợp với CNN Scratch; cuối cell in kích thước các tập và giải phóng dữ liệu trung gian để tiết kiệm RAM.

4.1.3. Xây dựng các lớp CNN và lan truyền ngược từ Scratch

```python
class Param:
    def __init__(self, value, decay=False):
        self.data = xp.asarray(value, dtype=xp.float32)
        self.grad = None
        self.m = self.v = None
        self.decay = decay


class Layer:
    def params(self):
        return []

    def buffers(self):
        return []

    def backward(self, g):
        raise NotImplementedError


# Chuyển các vùng ảnh thành hàng của ma trận để tính tích chập.
def unfold(x, k, stride, padding, pad_value=0):
    n, c, h, w = x.shape

    z = xp.pad(
        x,
        ((0, 0), (0, 0), (padding, padding), (padding, padding)),
        constant_values=pad_value
    ) if padding else x

    oh = (h + 2 * padding - k) // stride + 1
    ow = (w + 2 * padding - k) // stride + 1
    sn, sc, sh, sw = z.strides

    view = xp.lib.stride_tricks.as_strided(
        z,
        shape=(n, c, oh, ow, k, k),
        strides=(sn, sc, sh * stride, sw * stride, sh, sw)
    )

    cols = xp.ascontiguousarray(
        view.transpose(0, 2, 3, 1, 4, 5).reshape(
            n * oh * ow, c * k * k
        )
    )
    return cols, oh, ow
```

- Quản lý tham số: Lớp Param lưu trọng số, gradient và trạng thái phục vụ tối ưu; lớp Layer định nghĩa cấu trúc chung.
- Xây dựng các phép toán CNN: Tự cài đặt Conv2D, ReLU, MaxPooling, Flatten và Dense, kèm hàm forward và backward để tính đầu ra và gradient.
- Bổ sung các lớp hỗ trợ: Dropout giúp giảm quá khớp, BatchNorm chuẩn hóa đặc trưng và Global Average Pooling lấy trung bình theo không gian.
- Ghép mạng và khối residual: Sequential nối các lớp theo thứ tự; Bottleneck tạo nhánh tích chập và kết nối tắt để xây dựng ResNet50.

4.1.4. Thiết lập hàm mất mát, tối ưu và quy trình huấn luyện

```
EPOCHS = 20
BATCH_SIZE = 64
LR = 3e-4
MIN_LR = 3e-5
WEIGHT_DECAY = 1e-4

OUTPUT_DIR = Path("/content/cnn_scratch_results")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
RESULTS = {}


def cross_entropy(logits, y):
    shifted = logits - logits.max(axis=1, keepdims=True)
    ex = xp.exp(shifted)
    probs = ex / ex.sum(axis=1, keepdims=True)
    rows = xp.arange(len(y))

    loss = (
        xp.log(ex.sum(axis=1)) - shifted[rows, y]
    ).mean()

    grad = probs.copy()
    grad[rows, y] -= 1
    grad /= len(y)
    return loss, grad


def adamw_step(
    params, step, lr, weight_decay=1e-4, clip=5.0
):
    norm = xp.sqrt(
        sum(xp.sum(p.grad * p.grad) for p in params)
    )
    scale = xp.minimum(1.0, clip / (norm + 1e-8))

    for p in params:
        g = p.grad * scale
        if p.m is None:
            p.m = xp.zeros_like(p.data)
            p.v = xp.zeros_like(p.data)

        p.m *= 0.9
        p.m += 0.1 * g
        p.v *= 0.999
        p.v += 0.001 * g * g

        if p.decay:
```

- Cấu hình huấn luyện: Đặt 20 epoch, batch size 64 và learning rate giảm theo cosine từ 3e-4 xuống 3e-5.
- Tính loss và cập nhật trọng số: Tự xây dựng Cross-Entropy và gradient; sử dụng AdamW kết hợp weight decay và gradient clipping để hỗ trợ huấn luyện ổn định.
- Xây dựng quy trình dùng chung: Hàm run_experiment thực hiện huấn luyện, đánh giá validation và ghi lại loss, accuracy, thời gian của từng epoch.
- Lưu và đánh giá mô hình: Khôi phục trọng số có validation loss thấp nhất để đánh giá test bằng Accuracy, Macro Precision, Recall, F1 và ma trận nhầm lẫn; lưu kết quả rồi giải phóng GPU.

4.1.5. Xây dựng và huấn luyện mô hình CNN cơ bản

```
def make_basic():
    return Sequential(
        Conv2D(1, 16), ReLU(), MaxPool2D(),
        Conv2D(16, 32), ReLU(), MaxPool2D(),
        Flatten(),
        Dense(32 * 8 * 8, 64), ReLU(),
        Dense(64, 10)
    )

RESULTS["BasicCNN"] = run_experiment(
    "BasicCNN", make_basic
)
```

```
BasicCNN: 136,586 parameters

Epoch 01/20 | loss 0.5148/0.3638 | acc 81.83%/86.98% | 17.0s | lr 0.000300

Epoch 02/20 | loss 0.3419/0.3764 | acc 87.87%/86.32% | 8.4s | lr 0.000298
```

- **Trích xuất đặc trưng:** Sử dụng hai lớp tích chập với 16 và 32 bộ lọc, kết hợp ReLU để học đặc trưng của ảnh trang phục.
- **Giảm kích thước ảnh:** Hai lớp MaxPooling giảm kích thước không gian từ 32×32 xuống 8×8, tạo đầu ra gồm 32 bản đồ đặc trưng.
- **Phân loại:** Flatten chuyển đặc trưng thành vector 2.048 phần tử, sau đó đi qua Dense 64 nút và Dense 10 nút để tạo điểm dự đoán cho 10 lớp.
- **Huấn luyện và lưu kết quả:** Gọi run_experiment để huấn luyện, đánh giá và lưu kết quả vào RESULTS["BasicCNN"] phục vụ so sánh.

4.1.6. Xây dựng và huấn luyện mô hình VGG16

```
def make_vgg16():
    layers = []
    cin = 1

    for cout, count in [
        (64, 2),
        (128, 2),
        (256, 3),
        (512, 3),
        (512, 3)
    ]:
        for _ in range(count):
            layers += [Conv2D(cin, cout), ReLU()]
            cin = cout

        layers.append(MaxPool2D())

    layers += [
        Flatten(),
        Dense(512, 4096), ReLU(), Dropout(0.5),
        Dense(4096, 4096), ReLU(), Dropout(0.5),
        Dense(4096, 10)
    ]

    return Sequential(*layers)

RESULTS["VGG16"] = run_experiment(
    "VGG16", make_vgg16
)
```

- **Trích xuất đặc trưng:** Xây dựng 13 lớp tích chập chia thành 5 nhóm, với số bộ lọc tăng từ 64 lên 512, kết hợp ReLU.
- **Giảm kích thước đặc trưng:** MaxPooling sau mỗi nhóm giảm kích thước từ 32×32 xuống 1×1, tạo vector 512 phần tử sau Flatten.
- **Phân loại và giảm quá khớp:** Sử dụng ba lớp Dense theo cấu trúc **512 → 4096 → 4096 → 10**, kết hợp Dropout 0,5 ở hai lớp ẩn.
- **Huấn luyện và lưu kết quả:** Gọi run_experiment để huấn luyện, đánh giá và lưu kết quả vào RESULTS["VGG16"] phục vụ so sánh.

4.1.7. Xây dựng và huấn luyện mô hình ResNet50

```
def make_resnet50():
    layers = [
        Conv2D(1, 64, 7, 2, 3, False),
        BatchNorm2D(64),
        ReLU(),
        MaxPool2D(3, 2, 1)
    ]

    cin = 64

    for stage, (width, count) in enumerate([
        (64, 3),
        (128, 4),
        (256, 6),
        (512, 3)
    ]):
        for block in range(count):
            stride = 2 if stage > 0 and block == 0 else 1

            layers.append(
                Bottleneck(cin, width, stride)
            )
            cin = width * 4

    layers += [
        GlobalAvgPool(),
        Dense(2048, 10)
    ]

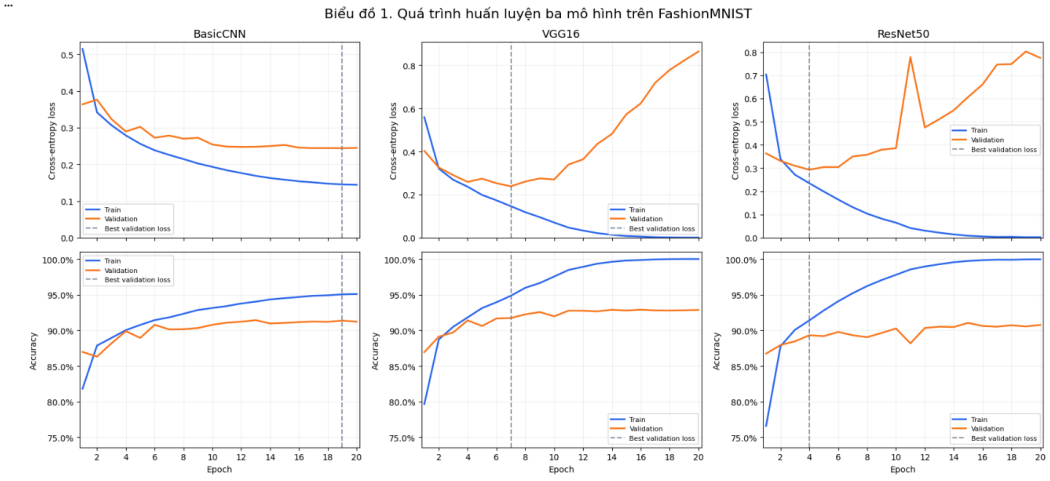
    return Sequential(*layers)

RESULTS["ResNet50"] = run_experiment(
    "ResNet50", make_resnet50
)
```

- **Xử lý đầu vào:** Sử dụng tích chập 7×7 với 64 bộ lọc, kết hợp BatchNorm, ReLU và MaxPooling để trích xuất đặc trưng và giảm kích thước ảnh.
- **Xây dựng các khối residual:** Tạo 4 nhóm Bottleneck với số khối **3–4–6–3**. Kết nối tắt giúp truyền thông tin và gradient qua mạng sâu.
- **Tổng hợp và phân loại:** Các nhóm tạo đầu ra có số kênh **256–512–1024–2048**. Global Average Pooling tạo vector 2.048 phần tử, sau đó Dense đưa ra điểm dự đoán cho 10 lớp.
- **Huấn luyện và lưu kết quả:** Gọi run_experiment để huấn luyện, đánh giá và lưu kết quả vào RESULTS["ResNet50"] phục vụ so sánh.

4.1.8.Kết quả thực nghiệm

a. Diễn biến Loss và Accuracy khi huấn luyện ba mô hình CNN



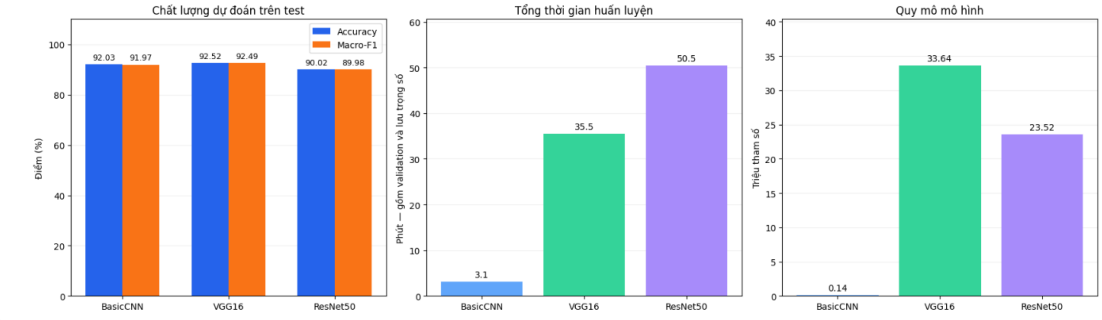
- **BasicCNN:** Loss giảm đều, độ chính xác validation đạt khoảng 91–92%. Khoảng cách giữa train và validation tăng nhẹ, cho thấy dấu hiệu quá khớp.

- **VGG16:** Accuracy train gần 100%, nhưng validation loss tăng sau epoch 7. Mô hình có biểu hiện quá khớp rõ rệt.
- **ResNet50:** Validation loss thấp nhất tại epoch 4, sau đó tăng và dao động; accuracy validation chủ yếu khoảng 90–91%.
- Trọng số được chọn tại **epoch có validation loss thấp nhất**, giúp hạn chế ảnh hưởng của quá khớp khi đánh giá trên tập test.

b. So sánh hiệu quả phân loại và chi phí huấn luyện của ba mô hình

	Mô hình	Accuracy (%)	Macro Precision (%)	Macro Recall (%)	Macro F1 (%)	Test loss	Best epoch	Thời gian train (phút)	Số tham số
0	BasicCNN	92.03	91.962	92.03	91.972	0.225	19	3.127	136586
1	VGG16	92.52	92.570	92.52	92.493	0.215	7	35.492	33637066
2	ResNet50	90.02	90.090	90.02	89.982	0.284	4	50.491	23522250

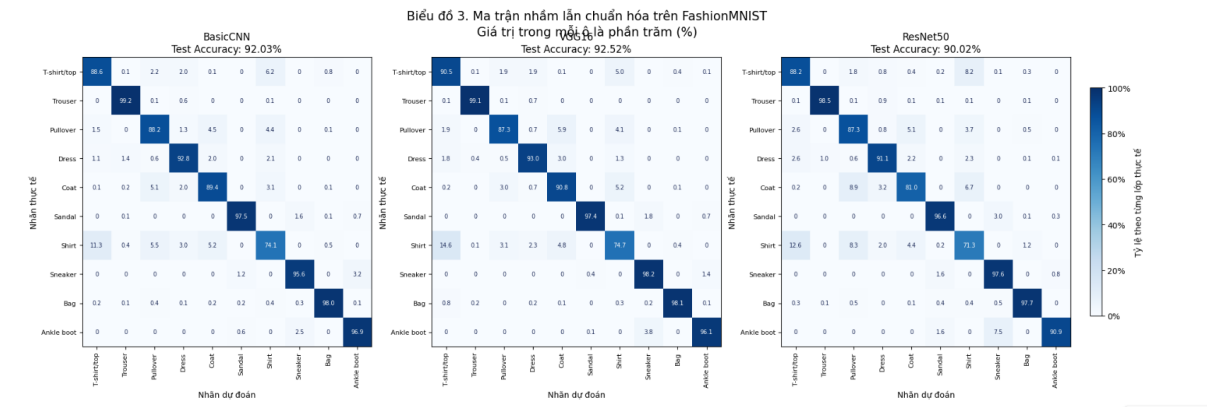
Biểu đồ 2. So sánh hiệu quả và chi phí của ba mô hình



- VGG16 đạt kết quả cao nhất: Accuracy 92,52%, Macro-F1 92,49% và test loss thấp nhất 0,215.
- BasicCNN đạt Accuracy 92,03%, chỉ thấp hơn VGG16 0,49 điểm phần trăm, trong khi thời gian huấn luyện khoảng 3,13 phút và có khoảng 136 nghìn tham số.
- ResNet50 đạt Accuracy 90,02%, nhưng mất khoảng 50,49 phút huấn luyện — lâu nhất trong ba mô hình. Trong thí nghiệm này, kiến trúc sâu hơn chưa mang lại kết quả tốt hơn.
- Nếu ưu tiên chất lượng phân loại, chọn VGG16; nếu ưu tiên mô hình nhỏ và huấn luyện nhanh, BasicCNN là lựa chọn phù hợp.

c. Ma trận nhầm lẫn của ba mô hình trên tập kiểm tra FashionMNIST

Biểu đồ 3. Ma trận nhầm lẫn chuẩn hóa trên FashionMNIST
Giá trị trong mỗi ô là phần trăm (%)

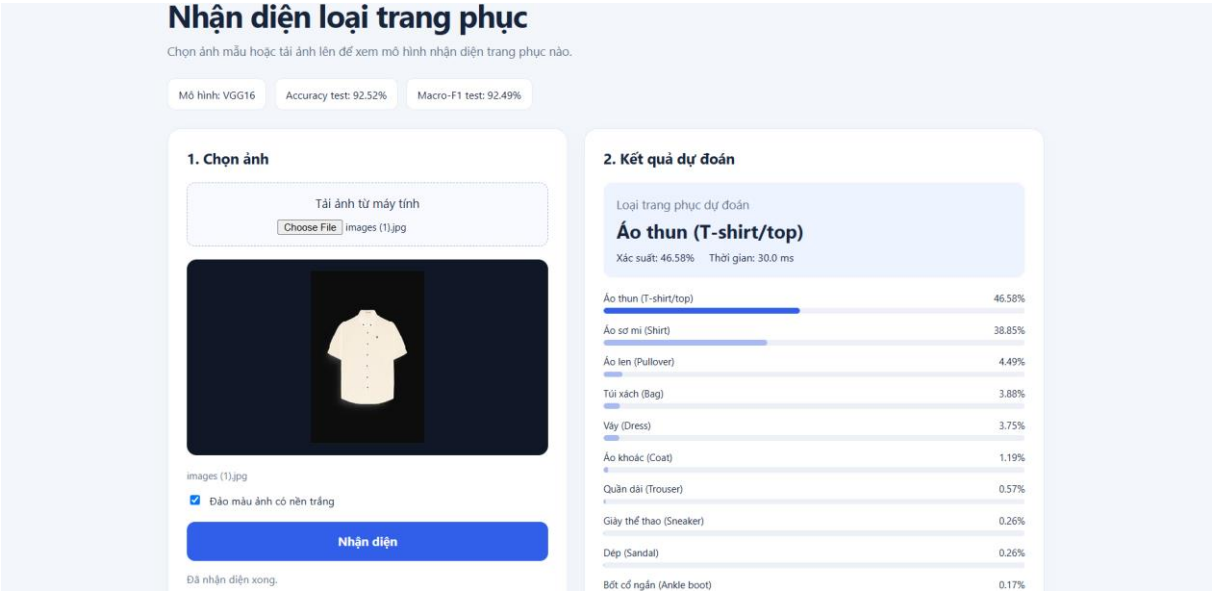


- Các ma trận được chuẩn hóa theo hàng; giá trị trên đường chéo thể hiện tỷ lệ nhận diện đúng của từng lớp.
- Các lớp Trouser, Sandal, Sneaker và Bag được cả ba mô hình nhận diện tốt, với tỷ lệ đúng trên 95%.

- Shirt là lớp khó phân loại nhất: BasicCNN đạt 74,1%, VGG16 74,7%, ResNet50 71,3%. Lớp này thường bị nhầm với T-shirt/top, có thể do hình dáng tương đồng.
- ResNet50 nhận diện Coat chỉ đạt 81,0% và Ankle boot đạt 86,9%, góp phần làm giảm độ chính xác tổng thể. VGG16 đạt kết quả chung cao nhất nhưng chưa tốt nhất ở mọi lớp.

Đánh giá mô hình VGG16 tối ưu nhất.

4.1.9.Deloy với mô hình VGG16



4.2. Dataset 2

4.2.1. Cài đặt môi trường và import các thư viện cần thiết

```
# ============================================================
# CELL 1: CÀI ĐẶT VÀ IMPORT THƯ VIỆN
# ============================================================

!pip -q install kagglehub

import os
import gc
import time
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import (
    EarlyStopping,
    ReduceLROnPlateau
)

from sklearn.model_selection import train_test_split

import kagglehub

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
TensorFlow: 2.21.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

4.2.2. Tải dataset từ kaggle

```
# ============================================================
# CELL 3: TẢI DATASET KUZUSHIJI TỪ KAGGLE
# ============================================================

dataset_path = kagglehub.dataset_download(
    "anokas/kuzushiji"
)

print("Dataset path:")
print(dataset_path)

print("\nCác file trong dataset:")

for root, dirs, files in os.walk(dataset_path):
    for file in files:
        if "kmnist" in file.lower():
            print(file)
```

4.2.3. Chia tập Train/Validation và trực quan hóa ảnh mẫu

```
# CELL 5: CHIA TRAIN / VALIDATION VÀ HIỂN THỊ ẢNH MẪU
# ============================================================

X_train, X_val, y_train, y_val = train_test_split(
    X_train_full,
    y_train_full,
    test_size=0.1,
    random_state=SEED,
    stratify=y_train_full
)

print("Train      :", X_train.shape)
print("Validation :", X_val.shape)
print("Test       :", X_test.shape)


plt.figure(figsize=(10, 4))

for i in range(15):

    plt.subplot(3, 5, i + 1)

    plt.imshow(
        X_train[i],
        cmap="gray"
    )

    plt.title(
        f"Class {y_train[i]}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Train      : (54000, 28, 28)
Validation : (6000, 28, 28)
Test       : (10000, 28, 28)
```

- Chia dữ liệu huấn luyện ban đầu thành 90% Train và 10% Validation bằng train_test_split, trong đó random_state=SEED giúp kết quả chia có thể lặp lại và stratify=y_train_full giữ tỷ lệ các lớp tương đối cân bằng giữa hai tập.
- Sau khi chia, chương trình in kích thước của ba tập Train, Validation và Test để kiểm tra số lượng mẫu trước khi đưa vào huấn luyện mô hình.

- Phần plt.imshow(..., cmap="gray") hiển thị 15 ảnh mẫu từ tập Train theo dạng ảnh xám, phù hợp với đặc trưng của Kuzushiji-MNIST.
- Mỗi ảnh được gắn nhãn Class tương ứng, giúp quan sát nhanh hình dạng dữ liệu và kiểm tra xem ảnh cùng nhãn đã được đọc, chia và hiển thị đúng hay chưa.

4.2.4. Chuẩn hóa và định dạng dữ liệu cho mô hình CNN cơ bản

```
# ============================================================
# CELL 6: CHUẨN BỊ DỮ LIỆU CHO CNN CƠ BẢN
# ============================================================

X_train_cnn = (
    X_train.astype("float32") / 255.0
)

X_val_cnn = (
    X_val.astype("float32") / 255.0
)

X_test_cnn = (
    X_test.astype("float32") / 255.0
)


X_train_cnn = np.expand_dims(
    X_train_cnn,
    axis=-1
)

X_val_cnn = np.expand_dims(
    X_val_cnn,
    axis=-1
)

X_test_cnn = np.expand_dims(
    X_test_cnn,
    axis=-1
)


print("Train:", X_train_cnn.shape)
print("Val  :", X_val_cnn.shape)
print("Test :", X_test_cnn.shape)
```

```
Train: (54000, 28, 28, 1)
Val  : (6000, 28, 28, 1)
Test : (10000, 28, 28, 1)
```

- Thực hiện chuẩn hóa giá trị pixel của ảnh từ khoảng 0–255 về 0–1 bằng cách chuyển dữ liệu sang float32 rồi chia cho 255.0. Việc chuẩn hóa giúp mô hình học ổn định hơn và quá trình tối ưu hội tụ nhanh hơn.
- Ba tập Train, Validation và Test đều được xử lý giống nhau để đảm bảo dữ liệu đầu vào có cùng thang giá trị trong toàn bộ quá trình huấn luyện và đánh giá.
- Hàm np.expand_dims(..., axis=-1) thêm một chiều kênh màu vào dữ liệu, biến ảnh từ dạng (28, 28) thành (28, 28, 1). Đây là định dạng phù hợp với đầu vào của lớp Conv2D đối với ảnh grayscale.
- Các lệnh print() cuối cell dùng để kiểm tra lại kích thước dữ liệu sau tiền xử lý, đảm bảo ba tập đã sẵn sàng để đưa vào mô hình CNN cơ bản.

4.2.5. Xây dựng, huấn luyện và đánh giá mô hình CNN cơ bản

```
tf.keras.backend.clear_session()

cnn_model = keras.Sequential([

    layers.Input(
        shape=(28, 28, 1)
    ),

    layers.Conv2D(
        32,
        3,
        padding="same",
        activation="relu"
    ),

    layers.MaxPooling2D(),

    layers.Conv2D(
        64,
        3,
        padding="same",
        activation="relu"
    ),

    layers.MaxPooling2D(),

    layers.Conv2D(
        128,
        3,
        padding="same",
        activation="relu"
    ),

    layers.GlobalAveragePooling2D(),

    layers.Dense(
        128,
        activation="relu"
    ),
```

- Mô hình CNN được xây dựng theo cấu trúc tăng dần số bộ lọc 32 → 64 → 128, giúp mô hình học từ các đặc trưng đơn giản như nét, cạnh đến các đặc trưng phức tạp hơn của ký tự Kuzushiji. Hai lớp MaxPooling2D giúp giảm kích thước đặc trưng và chi phí tính toán.
- GlobalAveragePooling2D chuyển các bản đồ đặc trưng cuối thành vector gọn hơn trước khi đi qua Dense(128). Dropout(0.3) loại ngẫu nhiên 30% neuron trong lúc huấn luyện nhằm hạn chế hiện tượng overfitting, còn lớp Dense(10, softmax) tạo xác suất dự đoán cho 10 lớp.
- Mô hình sử dụng Adam với learning rate 0.001 và hàm mất mát sparse_categorical_crossentropy, phù hợp với bài toán phân loại đa lớp khi nhãn được biểu diễn dưới dạng số nguyên từ 0 đến 9. EarlyStopping tự dừng khi validation loss không còn cải thiện, trong khi ReduceLROnPlateau giảm learning rate để mô hình tiếp tục tối ưu ổn định hơn.
- Quá trình huấn luyện tối đa 10 epoch, batch_size=256 nhằm tăng tốc trên GPU. Sau khi huấn luyện, mô hình được đánh giá trên tập Test thông qua Test Accuracy, Test

Loss và Training Time, từ đó làm cơ sở so sánh CNN cơ bản với VGG16 và ResNet50.

4.2.6. Xây dựng pipeline dữ liệu cho VGG16 và ResNet50

```
IMG_SIZE = 32
BATCH_SIZE = 256
AUTOTUNE = tf.data.AUTOTUNE


def preprocess_image(image, label):

    image = tf.cast(
        image,
        tf.float32
    )

    image = tf.expand_dims(
        image,
        axis=-1
    )

    image = tf.image.resize(
        image,
        [IMG_SIZE, IMG_SIZE]
    )

    image = tf.image.grayscale_to_rgb(
        image
    )

    return image, label


train_ds = (
    tf.data.Dataset
    .from_tensor_slices(
        (X_train, y_train)
    )
    .shuffle(
        10000,
        seed=SEED
    )
    .map(
        preprocess_image,
        num_parallel_calls=AUTOTUNE
```

- Chuyển dữ liệu Kuzushiji từ ảnh 28×28 grayscale sang định dạng 32×32×3, phù hợp với đầu vào của VGG16 và ResNet50. Hàm tf.image.resize() thay đổi kích thước ảnh, còn tf.image.grayscale_to_rgb() chuyển ảnh 1 kênh thành 3 kênh RGB.
- Dữ liệu được tổ chức bằng tf.data.Dataset, giúp quá trình đọc và xử lý dữ liệu hiệu quả hơn khi huấn luyện. Tập Train được shuffle() để xáo trộn thứ tự mẫu, hạn chế mô hình học theo thứ tự cố định của dữ liệu.
- batch(BATCH_SIZE) gom dữ liệu thành các lô 256 ảnh, còn prefetch(AUTOTUNE) cho phép chuẩn bị batch tiếp theo song song với quá trình huấn luyện, nhờ đó tận dụng GPU tốt hơn và rút ngắn thời gian train.
- Với tập Validation và Test, cache() được sử dụng để lưu dữ liệu đã xử lý trong bộ nhớ, tránh phải resize và chuyển RGB lặp lại qua nhiều epoch. Phần cuối kiểm tra kích thước một batch để xác nhận đầu vào đã có dạng (256, 32, 32, 3) trước khi đưa vào VGG16 và ResNet50.

4.2.7. Xây dựng và huấn luyện VGG16

```
tf.keras.backend.clear_session()
gc.collect()


vgg_base = keras.applications.VGG16(

    weights="imagenet",

    include_top=False,

    input_shape=(
        IMG_SIZE,
        IMG_SIZE,
        3
    )
)


# Không huấn luyện backbone
vgg_base.trainable = False


inputs = keras.Input(
    shape=(IMG_SIZE, IMG_SIZE, 3)
)


x = keras.applications.vgg16.preprocess_input(
    inputs
)


x = vgg_base(
    x,
    training=False
)
```

4.2.8.Xây dựng và huấn luyện Resnet50

```
tf.keras.backend.clear_session()
gc.collect()


resnet_base = keras.applications.ResNet50(

    weights="imagenet",

    include_top=False,

    input_shape=(
        IMG_SIZE,
        IMG_SIZE,
        3
    )
)


# Đóng băng backbone
resnet_base.trainable = False


inputs = keras.Input(
    shape=(IMG_SIZE, IMG_SIZE, 3)
)


x = keras.applications.resnet50.preprocess_input(
    inputs
)


x = resnet_base(
    x,
    training=False
)


x = layers.GlobalAveragePooling2D()(x)
```

4.2.9.Kết quả thực nghiệm

a. **Bảng so sánh hiệu năng của ba mô hình CNN Basic, VGG16 và ResNet50**

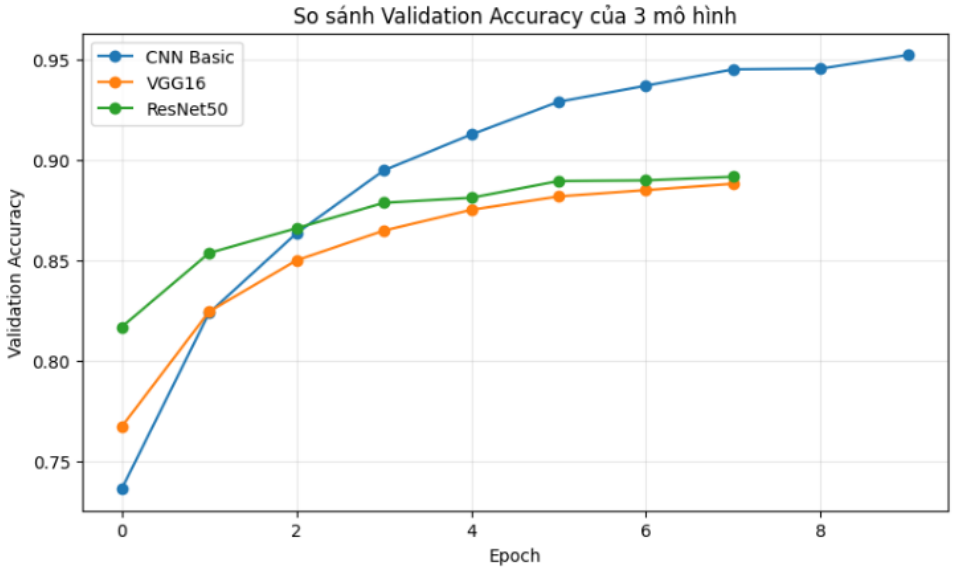
	Model	Test Accuracy	Test Loss	Training Time (s)
0	CNN Basic	0.8868	0.3794	40.30
1	VGG16	0.7992	0.6549	68.86
2	ResNet50	0.7941	0.6663	78.99

- CNN Basic đạt kết quả tốt nhất với Test Accuracy là 0.8868, cao hơn rõ rệt so với VGG16 (0.7992) và ResNet50 (0.7941).
- CNN Basic cũng có Test Loss thấp nhất, 0.3794, cho thấy mô hình không chỉ dự đoán đúng nhiều hơn mà còn có mức sai số thấp hơn hai mô hình còn lại.

- Về thời gian huấn luyện, CNN Basic chỉ mất khoảng 40,30 giây, trong khi VGG16 mất 68,86 giây và ResNet50 mất 78,99 giây. Như vậy CNN Basic vừa chính xác hơn vừa tiết kiệm thời gian tính toán hơn.
- Kết quả này cho thấy với dữ liệu Kuzushiji-MNIST kích thước nhỏ 28×28, một kiến trúc CNN đơn giản có thể phù hợp hơn các mạng sâu pretrained như VGG16 và ResNet50, vốn được thiết kế chủ yếu cho ảnh RGB tự nhiên có kích thước lớn.

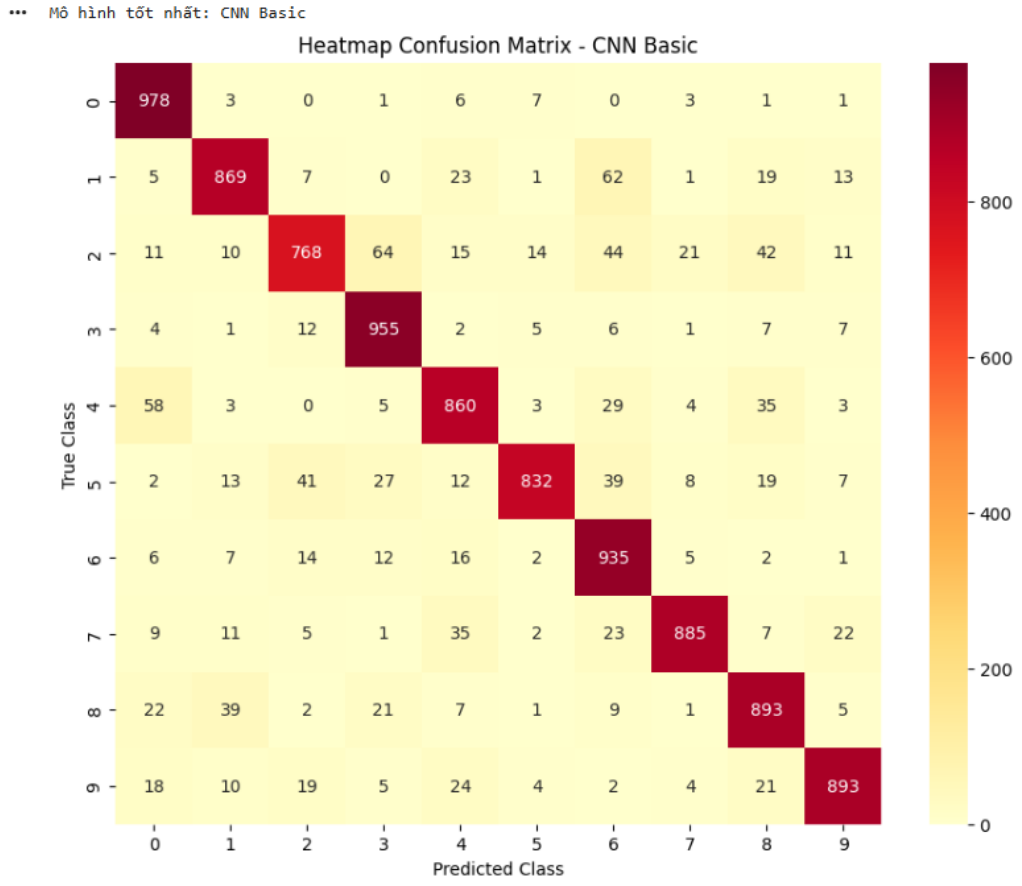
b. So sánh Validation Accuracy của CNN Basic, VGG16 và ResNet50 qua các epoch

...



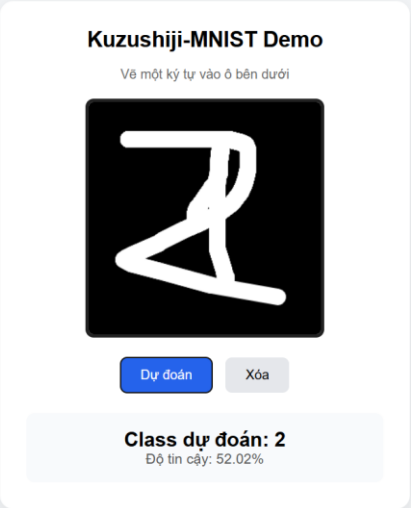
- Cả ba mô hình đều có **Validation Accuracy tăng dần qua các epoch**, cho thấy quá trình huấn luyện diễn ra đúng hướng và các mô hình đều học được đặc trưng từ dữ liệu.
- **CNN Basic cải thiện mạnh nhất**, từ khoảng **0,74 ở epoch đầu** lên trên **0,95** ở epoch cuối. Sau khoảng epoch 2–3, CNN Basic bắt đầu vượt rõ VGG16 và ResNet50.
- VGG16 và ResNet50 tăng nhanh trong những epoch đầu nhưng sau đó có xu hướng **bão hòa quanh mức 0,89**, cho thấy hiệu quả cải thiện về sau khá hạn chế.
- ResNet50 có Validation Accuracy ban đầu cao hơn hai mô hình còn lại, nhưng CNN Basic lại có khả năng tiếp tục học và cải thiện tốt hơn. Điều này củng cố nhận định rằng mô hình phức tạp hơn chưa chắc mang lại kết quả tốt hơn trên tập ảnh nhỏ như Kuzushiji-MNIST.

c. Heatmap Confusion Matrix của mô hình CNN Basic



- Ma trận nhầm lẫn cho thấy phần lớn giá trị lớn tập trung trên **đường chéo chính**, nghĩa là CNN Basic phân loại đúng phần lớn các mẫu thuộc 10 lớp.
- Một số lớp được nhận diện rất tốt, chẳng hạn **lớp 0 có 978 mẫu dự đoán đúng**, lớp 3 có **955 mẫu**, lớp 6 có **935 mẫu**. Đây là những lớp mà mô hình học đặc trưng khá rõ ràng.
- Một số cặp lớp vẫn bị nhầm đáng kể. Ví dụ lớp 2 bị nhầm sang lớp 3 **64 lần**, lớp 1 bị nhầm sang lớp 6 **62 lần**, lớp 4 bị nhầm sang lớp 0 **58 lần**. Điều này cho thấy một số ký tự Kuzushiji có hình dạng tương đối giống nhau.
- Nhìn tổng thể, số lượng dự đoán đúng vẫn chiếm ưu thế lớn so với các ô ngoài đường chéo. Điều này phù hợp với Test Accuracy khoảng **88,68%**, đồng thời cho thấy mô hình CNN Basic có khả năng phân loại khá tốt nhưng vẫn còn dư địa cải thiện ở các lớp có hình dạng tương đồng.

4.2.9.Deloy trên mô hình tố nhất



CHƯƠNG 4. RNN

1.Tổng quan kiến thức về RNN

- Khái niệm: RNN (Recurrent Neural Network) là mạng nơ-ron hồi tiếp, được thiết kế để xử lý dữ liệu có thứ tự như văn bản, âm thanh và chuỗi thời gian. RNN sử dụng thông tin từ các bước trước để hỗ trợ xử lý bước hiện tại, nhờ đó có thể học mối quan hệ giữa các phần tử trong chuỗi.
- Các thành phần chính: RNN gồm đầu vào tại từng bước, trạng thái ẩn và đầu ra. Trạng thái ẩn đóng vai trò lưu giữ thông tin về những phần tử đã xử lý. Các trọng số được dùng chung qua các bước thời gian, giúp mô hình xử lý chuỗi có độ dài khác nhau.
- Nguyên lý hoạt động: Tại mỗi bước, RNN kết hợp đầu vào hiện tại với trạng thái ẩn của bước trước để tạo trạng thái ẩn mới. Mô hình có thể đưa ra dự đoán ở từng bước hoặc sau khi đọc toàn bộ chuỗi. Khi huấn luyện, RNN sử dụng lan truyền ngược qua thời gian (BPTT) để tính gradient và cập nhật trọng số.
- Ưu điểm: RNN khai thác được thứ tự và sự phụ thuộc giữa các phần tử, phù hợp với dữ liệu tuần tự. Việc chia sẻ trọng số giúp số lượng tham số không tăng theo độ dài chuỗi. Mô hình có thể áp dụng cho nhiều dạng bài toán như phân loại cả chuỗi hoặc dự đoán giá trị tiếp theo.
- Hạn chế: RNN cơ bản khó ghi nhớ thông tin ở những bước cách xa nhau do hiện tượng tiêu biến hoặc bùng nổ gradient. Việc xử lý lần lượt theo thời gian cũng hạn chế khả năng tính toán song song. Vì vậy, mô hình có thể huấn luyện chậm và kém hiệu quả với chuỗi dài.
- Ứng dụng và mô hình mở rộng: RNN được sử dụng trong phân tích cảm xúc, dự báo chuỗi thời gian, nhận dạng tiếng nói và mô hình hóa hành vi khách hàng. Các biến thể như LSTM và GRU giúp học quan hệ dài hạn tốt hơn; RNN hai chiều kết hợp thông tin từ hai hướng của chuỗi; Attention giúp mô hình tập trung vào những phần thông tin quan trọng khi dự đoán.

2.Giới thiệu các dataset

a. SJC Gold Price Vietnam

Link: https://www.kaggle.com/datasets/hoangziet/gold-price-in-10-years

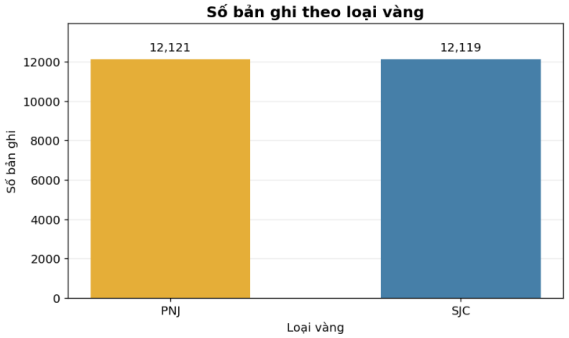
1. Kích cỡ, phạm vi thời gian và số lượng bản ghi của dataset

TỔNG QUAN DATASET GIÁ VÀNG VIỆT NAM

Kích cỡ và phạm vi dữ liệu

Thông tin	Giá trị
Số bản ghi	24,240
Số cột	4
Số loại vàng	2
Số ngày có dữ liệu	3,042
Khoảng thời gian	23/06/2014 – 18/06/2024
Dung lượng CSV	899.6 KB

Số bản ghi theo loại vàng



- Dataset gồm 24.240 bản ghi và 4 cột, cung cấp thông tin về loại vàng, hai cột giá và thời điểm cập nhật. Dung lượng khoảng 899,6 KB, thuận tiện để xử lý và phân tích trên Google Colab.
- Dữ liệu trải dài từ 23/06/2014 đến 18/06/2024, với 3.042 ngày có dữ liệu. Khoảng thời gian gần 10 năm tạo điều kiện nghiên cứu biến động giá ở nhiều giai đoạn.
- Số bản ghi của hai loại vàng gần như bằng nhau: PNJ có 12.121 bản ghi, SJC có 12.119 bản ghi, chỉ chênh lệch 2 bản ghi. Điều này thuận lợi cho việc so sánh hai chuỗi dữ liệu.
- Số bản ghi lớn hơn nhiều số ngày vì có hai loại vàng và nhiều lần cập nhật trong ngày. Khi xây dựng chuỗi giá theo ngày cho RNN, cần chọn một quy tắc thống nhất, chẳng hạn lấy lần cập nhật cuối ngày.

2. Một số mẫu giá vàng PNJ và SJC tại các thời điểm khác nhau

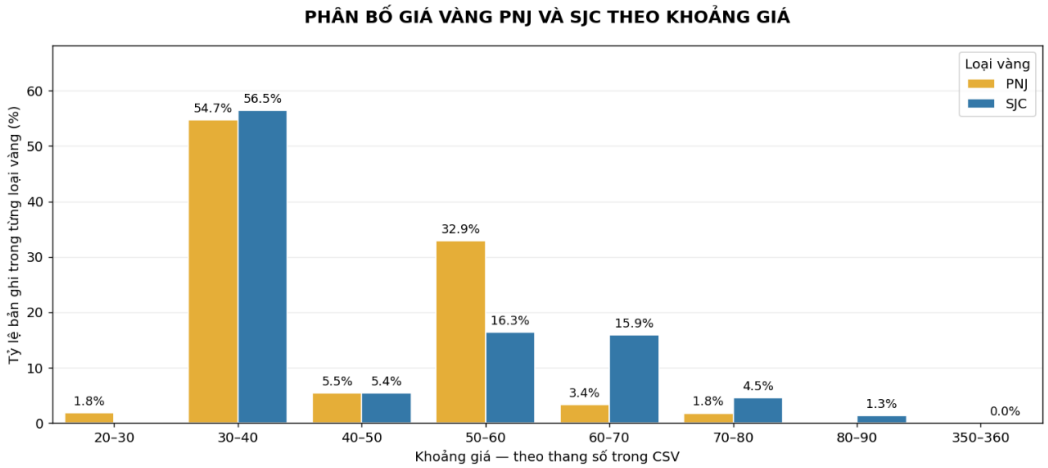
MỘT SỐ MẪU GIÁ VÀNG PNJ VÀ SJC QUA CÁC THỜI ĐIỂM

Loại vàng	Ngày giờ cập nhật	buy (CSV)	sell (CSV)
PNJ	23/06/2014 08:18:07	34.300	34.100
PNJ	01/11/2016 08:13:11	34.990	34.590
PNJ	09/11/2020 10:58:02	55.000	54.500
PNJ	18/06/2024 08:09:01	75.000	73.300
SJC	23/06/2014 08:18:07	36.840	36.720
SJC	01/11/2016 17:41:10	35.950	35.750
SJC	09/11/2020 15:21:57	56.950	56.500
SJC	18/06/2024 08:09:01	76.980	74.980

- Bảng minh họa cấu trúc dữ liệu với bốn thông tin: loại vàng, ngày giờ cập nhật, cột buy và cột sell. Thời gian được ghi đến giây, cho phép xác định thứ tự các lần cập nhật.
- Các mẫu cho thấy mức giá năm 2024 cao hơn rõ rệt so với năm 2014. Chẳng hạn, giá tại cột sell của PNJ tăng từ 34.100 lên 73.300, còn SJC tăng từ 36.720 lên 74.980, theo cách ghi trong CSV. Đây là so sánh giữa các mẫu được chọn, chưa phản ánh toàn bộ diễn biến trong giai đoạn.
- Trong những mẫu hiển thị, giá SJC cao hơn PNJ ở cả hai cột giá. Tuy nhiên, cần kiểm tra toàn bộ dữ liệu để kết luận mức chênh lệch giữa hai loại vàng.

- Tất cả mẫu đều có buy lớn hơn sell, nên cần đối chiếu nguồn để xác minh ý nghĩa hai cột trước khi gọi chúng là giá mua vào và bán ra trong báo cáo.

3. Phân bố tỷ lệ bản ghi của PNJ và SJC theo khoảng giá

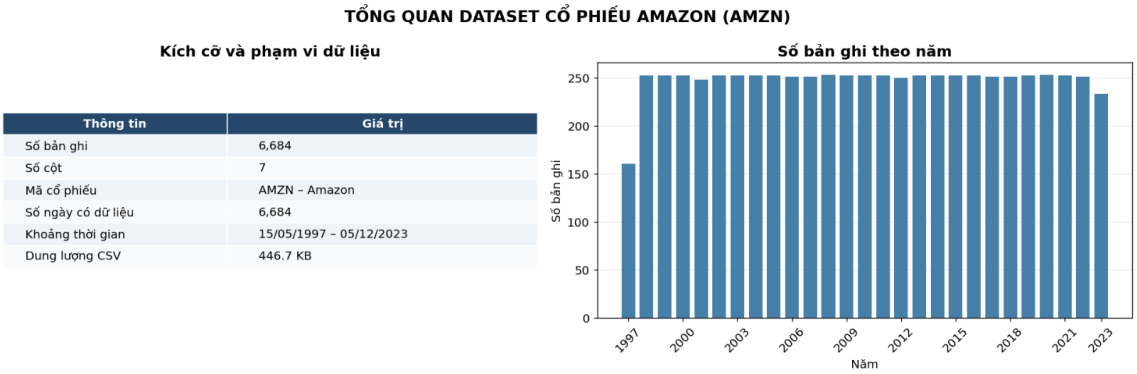


- Cả hai loại vàng tập trung nhiều nhất ở khoảng **30–40**: PNJ chiếm **54,7%**, SJC chiếm **56,5%**. Đây là khoảng giá phổ biến nhất trong các bản ghi của mỗi loại.
- Ở khoảng **50–60**, PNJ chiếm **32,9%**, cao hơn đáng kể so với **16,3%** của SJC. Ngược lại, tại khoảng **60–70**, SJC đạt **15,9%**, trong khi PNJ chỉ chiếm **3,4%**.
- SJC có tỷ lệ bản ghi ở các khoảng **70–80 và 80–90** cao hơn PNJ. Điều này cho thấy phân bố của SJC có tỷ trọng lớn hơn ở các khoảng giá cao trong dataset.
- Khoảng **350–360** nằm rất xa phần lớn dữ liệu và cần được kiểm tra về giá trị, định dạng hoặc đơn vị. Nhãn **0,0%** có thể do tỷ lệ rất nhỏ được làm tròn, không đồng nghĩa hoàn toàn không có bản ghi.
- Biểu đồ thể hiện **tỷ lệ bản ghi cập nhật**, không phải tỷ lệ số ngày. Nó giúp so sánh phân bố giá, nhưng chưa thể hiện thời điểm giá tăng hoặc giảm.

b. Amazon Stock Price (AMZN)

Link: https://www.kaggle.com/datasets/henryshan/amazon-com-inc-amzn/data

1. Kích cỡ, phạm vi thời gian và số lượng bản ghi theo năm



- Dataset gồm 6.684 bản ghi và 7 cột, có dung lượng khoảng 446,7 KB. Dữ liệu tương đối gọn, thuận tiện để phân tích và huấn luyện các mô hình RNN trên Google Colab.
- Khoảng thời gian từ 15/05/1997 đến 05/12/2023, trải dài hơn 26 năm. Lịch sử dài giúp nghiên cứu giá cổ phiếu qua nhiều giai đoạn và xây dựng các cửa sổ dữ liệu để dự đoán.
- Số ngày có dữ liệu bằng số bản ghi, cho thấy mỗi ngày được ghi nhận một bản ghi và không có ngày trùng lặp.
- Phần lớn các năm có khoảng 250 bản ghi, phù hợp với dữ liệu theo phiên giao dịch thay vì tất cả ngày trong năm. Năm 1997 và 2023 có ít bản ghi hơn vì dataset chỉ bao phủ một phần của hai năm này.

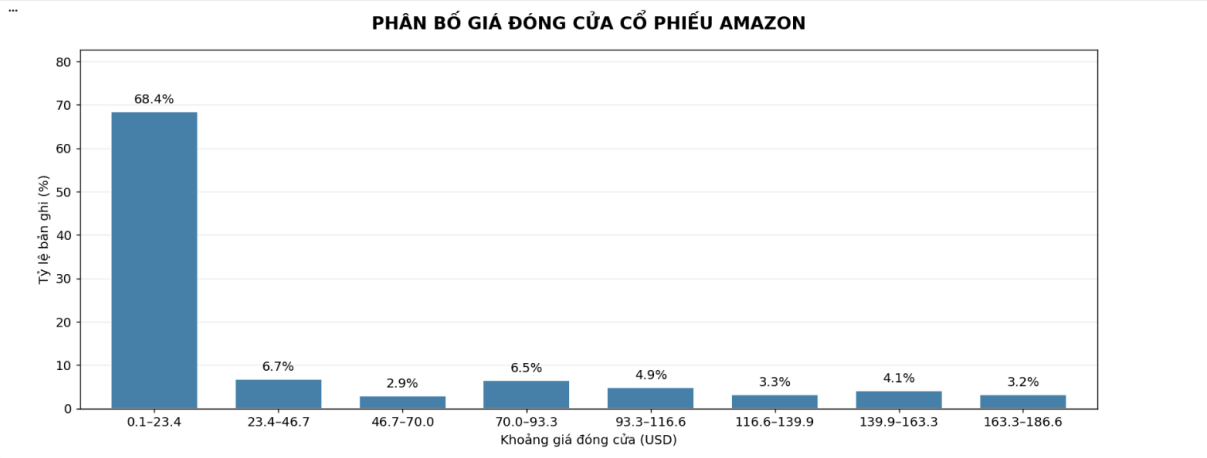
2. Một số mẫu dữ liệu cổ phiếu Amazon qua các thời điểm

MỘT SỐ MẪU DỮ LIỆU CỔ PHIẾU AMAZON QUA CÁC THỜI ĐIỂM

Ngày	Open	High	Low	Close	Adj Close	Volume
15/05/1997	0.1219	0.1250	0.0964	0.0979	0.0979	1,443,120,000
27/02/2001	0.5906	0.6062	0.5813	0.5875	0.5875	154,652,000
16/12/2004	2.0430	2.0500	1.9855	2.0025	2.0025	108,298,000
02/10/2008	3.3815	3.4480	3.2705	3.3680	3.3680	174,522,000
17/07/2012	10.8725	10.8970	10.6980	10.8465	10.8465	39,740,000
04/05/2016	33.1295	33.7000	33.1070	33.5450	33.5450	92,710,000
20/02/2020	108.6535	108.8395	106.3725	107.6550	107.6550	62,626,000
05/12/2023	143.5500	148.5700	143.1300	146.8800	146.8800	46,754,800

- Bảng thể hiện đầy đủ các thuộc tính: ngày giao dịch, giá mở cửa, giá cao nhất, giá thấp nhất, giá đóng cửa, giá đóng cửa điều chỉnh và khối lượng giao dịch. Những thông tin này cho phép phân tích cả mức giá và hoạt động giao dịch.
- Các mẫu cho thấy giá đóng cửa ở những thời điểm sau cao hơn rõ rệt: từ khoảng 0,0979 USD năm 1997, lên 33,5450 USD năm 2016, 107,6550 USD năm 2020 và 146,8800 USD năm 2023. Đây là xu hướng trong các mẫu được chọn, chưa chứng minh giá tăng liên tục trên toàn bộ chuỗi.
- Giá mở cửa và đóng cửa có thể khác nhau trong cùng một phiên. Ví dụ, ngày 05/12/2023, giá mở cửa là 143,55 USD, còn giá đóng cửa là 146,88 USD, cho thấy mức tăng trong phiên đó.
- Trong các mẫu hiển thị, Close và Adj Close bằng nhau. Tuy nhiên, cần kiểm tra toàn bộ dữ liệu và cách điều chỉnh chia tách cổ phiếu trước khi diễn giải các mức giá lịch sử rất thấp.
- Khối lượng giao dịch thay đổi đáng kể giữa các mẫu và có thang số lớn hơn nhiều so với giá. Nếu dùng đồng thời giá và khối lượng làm đầu vào RNN, cần chuẩn hóa các đặc trưng.

3. Phân bố tỷ lệ bản ghi theo khoảng giá đóng cửa



- Phân bố tập trung mạnh ở khoảng 0,1–23,4 USD, chiếm 68,4% tổng số bản ghi. Như vậy, hơn hai phần ba số phiên trong dataset thuộc khoảng giá thấp nhất.
- Các khoảng còn lại có tỷ lệ nhỏ hơn nhiều. Khoảng 23,4–46,7 USD chiếm 6,7%, khoảng 70,0–93,3 USD chiếm 6,5%; các khoảng khác dao động khoảng 2,9–4,9%.
- Phân bố có dạng lệch phải, với nhiều giá trị thấp và phần dữ liệu còn lại trải rộng đến khoảng 186,6 USD. Điều này cho thấy mức giá trên toàn bộ lịch sử không phân bố đồng đều.
- Khi đánh giá mô hình, nên xem thêm sai số theo từng giai đoạn hoặc khoảng giá để biết mô hình dự đoán tốt ở đâu, vì một kết quả tổng hợp có thể che khuất khác biệt giữa các nhóm.
- Biểu đồ chỉ thể hiện tần suất xuất hiện của các mức giá, chưa cho biết chúng xuất hiện vào thời điểm nào. Muốn nhận xét diễn biến tăng, giảm hoặc biến động theo thời gian cần bổ sung biểu đồ đường.

3. Các mô hình huấn luyện RNN

3.1. RNN cơ bản — Recurrent Neural Network

- **Khái niệm:** Mạng nơ-ron hồi tiếp xử lý dữ liệu dạng chuỗi, trong đó thông tin từ bước trước được truyền sang bước hiện tại thông qua trạng thái ẩn.
- **Cơ chế hoạt động:** Ở mỗi bước thời gian, RNN kết hợp đầu vào hiện tại với trạng thái ẩn trước đó để tạo trạng thái ẩn mới; các bước sử dụng chung bộ trọng số.
- **Ưu điểm:** Kiến trúc đơn giản, dễ triển khai, số lượng tham số tương đối ít; phù hợp để học các quan hệ ngắn hạn trong chuỗi.
- **Hạn chế:** Dễ gặp hiện tượng tiêu biến hoặc bùng nổ gradient khi huấn luyện, gây khó khăn trong việc học các phụ thuộc dài hạn.
- **Ứng dụng trong bài:** Đọc chuỗi giá vàng của các phiên trước để dự đoán giá phiên tiếp theo, làm mô hình nền để so sánh với hai mô hình cải tiến.

3.2. LSTM — Long Short-Term Memory

- **Khái niệm:** Biến thể của RNN được thiết kế để hỗ trợ ghi nhớ thông tin dài hạn thông qua trạng thái ô nhớ.
- **Cơ chế hoạt động:** Sử dụng ba cổng chính: **cổng quên** loại bỏ thông tin không cần thiết, **cổng đầu vào** cập nhật thông tin mới và **cổng đầu ra** điều khiển thông tin tạo thành trạng thái ẩn.
- **Ưu điểm:** Giảm khó khăn do tiêu biến gradient, hỗ trợ học quan hệ giữa các bước cách xa nhau và lựa chọn thông tin cần lưu giữ.
- **Hạn chế:** Có nhiều tham số và phép tính hơn RNN cơ bản; thời gian huấn luyện thường dài hơn và vẫn có nguy cơ học quá khớp.
- **Ứng dụng trong bài:** Học các quan hệ theo thời gian trong lịch sử giá vàng, từ đó đánh giá lợi ích của cơ chế ô nhớ so với RNN cơ bản.

3.3. Stacked LSTM — LSTM nhiều tầng

- **Khái niệm:** Kiến trúc gồm nhiều tầng LSTM xếp chồng lên nhau, giúp mạng học biểu diễn dữ liệu chuỗi ở nhiều mức độ.
- **Cơ chế hoạt động:** Tầng LSTM đầu tiên nhận chuỗi giá vàng; chuỗi trạng thái ẩn của tầng này trở thành đầu vào cho tầng LSTM tiếp theo.
- **Ưu điểm:** Tăng khả năng biểu diễn của mạng, cho phép các tầng sau học những quan hệ phức tạp hơn từ đặc trưng mà tầng trước tạo ra.
- **Hạn chế:** Tốn tài nguyên hơn, khó điều chỉnh hơn và dễ học quá khớp; tăng số tầng không bảo đảm độ chính xác cao hơn.
- **Ứng dụng trong bài:** Sử dụng hai tầng LSTM để dự đoán giá vàng, qua đó kiểm tra việc tăng độ sâu có cải thiện sai số so với LSTM một tầng hay không.

4.Thực nghiệm trên các tập dataset

4.1. Dataset 1

4.1.1. Khởi tạo môi trường và thiết lập tham số cho bài toán dự đoán giá vàng SJC

```python
import random
import time
from pathlib import Path

import kagglehub
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

from IPython.display import display


SEED = 42

GOLD_TYPE = "SJC"
TARGET_COL = "sell"

# Bỏ 50% số ngày đầu, chỉ giữ nửa sau.
KEEP_FROM = 0.50

WINDOW_SIZE = 30
BATCH_SIZE = 64
HIDDEN_SIZE = 64

EPOCHS = 60
```

4.1.2. Làm sạch dữ liệu, tổng hợp giá vàng SJC theo ngày và lựa chọn nửa sau chuỗi

```
working_df = raw_df.copy()

working_df["type"] = (
    working_df["type"]
    .str.strip()
    .str.upper()
)

working_df["Timestamp"] = pd.to_datetime(
    working_df["update"],
    format="%d/%m/%Y %H:%M:%S",
    errors="coerce"
)

for column in ["buy", "sell"]:
    working_df[column] = pd.to_numeric(
        working_df[column].str.strip(),
        errors="coerce"
    )

selected_df = working_df.loc[
    working_df["type"].eq(GOLD_TYPE)
].copy()

selected_df = selected_df.dropna(
    subset=["Timestamp", TARGET_COL]
)

selected_df = selected_df.loc[
    np.isfinite(selected_df[TARGET_COL])
    & selected_df[TARGET_COL].gt(0)
].copy()

selected_df = selected_df.sort_values(
    "Timestamp",
    kind="stable"
)
```

- Chuẩn hóa và làm sạch dữ liệu: Sao chép dữ liệu gốc để xử lý, chuẩn hóa tên loại vàng, chuyển thời gian cập nhật sang kiểu datetime và chuyển giá mua, giá bán sang dạng số. Sau khi chọn SJC, loại bỏ những bản ghi có thời gian hoặc giá mục tiêu không hợp lệ, giá không hữu hạn hoặc không lớn hơn 0.
- Tạo chuỗi giá theo ngày: Sắp xếp bản ghi theo thời gian cập nhật, sau đó giữ bản ghi hợp lệ cuối cùng của mỗi ngày. Cột giá mục tiêu được đổi tên thành Price, tạo bảng gồm Date và Price. Các ngày không có dữ liệu không được tự bổ sung.
- Chỉ giữ nửa sau chuỗi dữ liệu: Với KEEP_FROM = 0.50, chương trình bỏ 50% số ngày đầu và giữ phần còn lại theo thứ tự thời gian. Cách chọn này tập trung vào giai đoạn gần cuối dataset, đồng thời giảm lượng dữ liệu cần xử lý; đây là chia theo số ngày có dữ liệu, không phải đúng một nửa khoảng thời gian lịch.
- Kiểm tra và mô tả dữ liệu sau xử lý: Kiểm tra lượng dữ liệu tối thiểu, thứ tự ngày, tính duy nhất của ngày và tính hữu hạn của giá. Cuối cùng, hiển thị số ngày đã bỏ, số ngày giữ lại, khoảng thời gian, thống kê giá và các bản ghi đầu–cuối để xác nhận dữ liệu trước khi chia tập và tạo cửa sổ.

4.1.3. Xây dựng và huấn luyện mô hình RNN cơ bản để dự đoán giá vàng SJC

```
class GoldRNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.rnn = nn.RNN(
            input_size=1,
            hidden_size=HIDDEN_SIZE,
            num_layers=1,
            nonlinearity="tanh",
            batch_first=True
        )

        self.fc = nn.Linear(
            HIDDEN_SIZE,
            1
        )

    def forward(self, x):
        _, hidden_state = self.rnn(x)

        final_hidden = hidden_state[-1]

        return self.fc(final_hidden)


set_seed()

rnn_model = GoldRNN()

print(rnn_model)

rnn_model = train_model(
    "RNN",
    rnn_model
)
```

4.1.4. Xây dựng và huấn luyện mô hình LSTM một tầng để dự đoán giá vàng SJC

```
class GoldLSTM(nn.Module):
    def __init__(self):
        super().__init__()

        self.lstm = nn.LSTM(
            input_size=1,
            hidden_size=HIDDEN_SIZE,
            num_layers=1,
            batch_first=True
        )

        self.fc = nn.Linear(
            HIDDEN_SIZE,
            1
        )

    def forward(self, x):
        _, (hidden_state, cell_state) = self.lstm(x)

        final_hidden = hidden_state[-1]

        return self.fc(final_hidden)


set_seed()

lstm_model = GoldLSTM()

print(lstm_model)

lstm_model = train_model(
    "LSTM",
    lstm_model
)
```

4.1.5. Xây dựng và huấn luyện mô hình Stacked LSTM hai tầng để dự đoán giá vàng SJC

```python
class GoldStackedLSTM(nn.Module):
    def __init__(self):
        super().__init__()

        self.lstm = nn.LSTM(
            input_size=1,
            hidden_size=HIDDEN_SIZE,
            num_layers=2,
            dropout=0.2,
            batch_first=True
        )

        self.fc = nn.Linear(
            HIDDEN_SIZE,
            1
        )

    def forward(self, x):
        _, (hidden_state, cell_state) = self.lstm(x)

        # Trạng thái cuối của tầng trên cùng.
        final_hidden = hidden_state[-1]

        return self.fc(final_hidden)


set_seed()

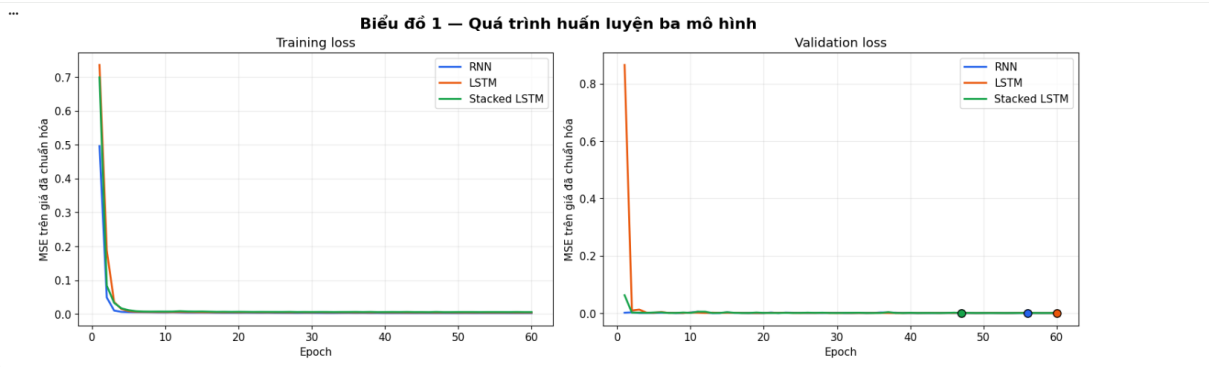
stacked_lstm_model = GoldStackedLSTM()

print(stacked_lstm_model)

stacked_lstm_model = train_model(
    "Stacked LSTM",
    stacked_lstm_model
)
```

4.1.6.Kết quả thực nghiệm

a. So sánh quá trình huấn luyện của RNN, LSTM và Stacked LSTM

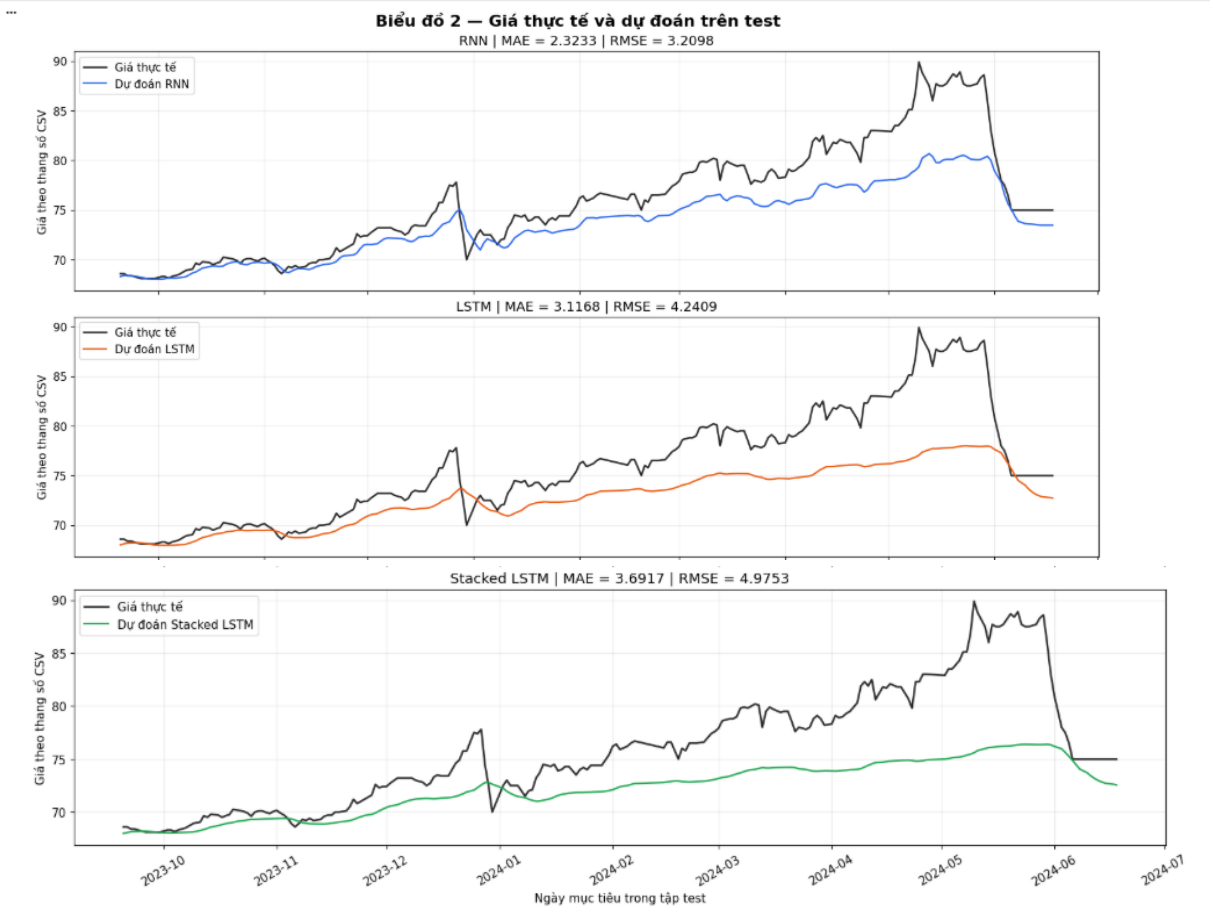


- **Cả ba mô hình hội tụ nhanh:** Training loss giảm mạnh trong khoảng 5–10 epoch đầu, sau đó duy trì ở mức thấp đến epoch 60. Điều này cho thấy các mô hình đã học được những quy luật cơ bản của chuỗi giá trong tập huấn luyện.
- **LSTM có sai số ban đầu cao nhất:** Validation loss của LSTM ở epoch đầu khoảng 0,86, cao hơn RNN và Stacked LSTM, nhưng giảm nhanh ở các epoch tiếp theo. Sai số khởi đầu cao không đồng nghĩa mô hình sẽ có kết quả cuối cùng kém nhất.
- **Giai đoạn cuối tương đối ổn định:** Các đường training loss và validation loss đều thấp, không xuất hiện xu hướng validation loss tăng mạnh kéo dài. Tuy nhiên, thang

trục lớn khiến những khác biệt nhỏ khó quan sát; biểu đồ này chưa đủ để kết luận mô hình nào tổng quát hóa tốt nhất.

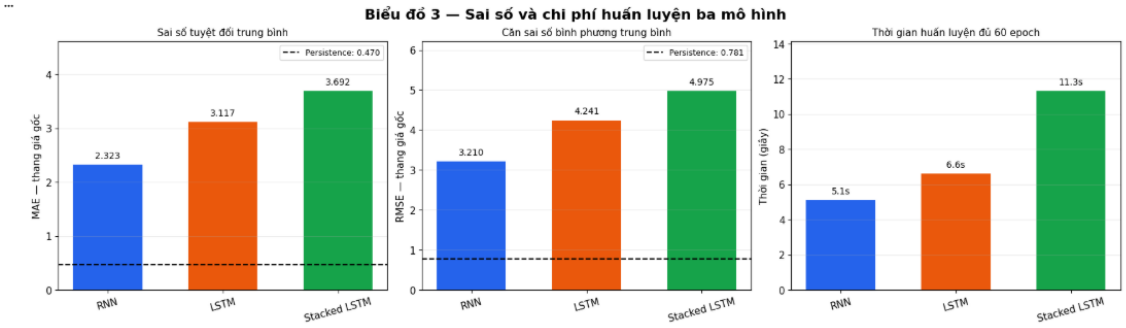
- **Loss thấp cần được hiểu đúng:** Giá trị ở đây là MSE trên dữ liệu đã chuẩn hóa, nên không thể diễn giải trực tiếp thành mức sai lệch giá vàng. Các dấu chấm đánh dấu epoch có validation loss thấp nhất của từng mô hình, dùng để chọn trọng số đánh giá.

b. So sánh giá vàng SJC thực tế và giá dự đoán của ba mô hình trên tập kiểm tra



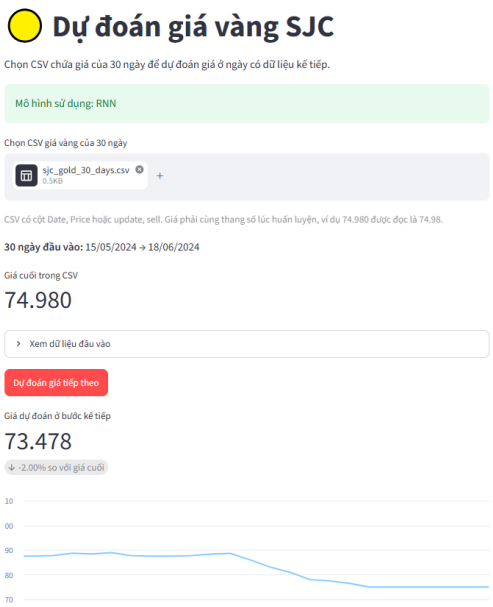
- Các mô hình nhận diện được xu hướng chung: Đường dự đoán đều phản ánh xu hướng tăng từ cuối năm 2023 đến tháng 5/2024 và giảm vào cuối chuỗi. Ở giai đoạn đầu, khi giá biến động ít, dự đoán tương đối gần giá thực tế.
- RNN bám sát giá thực tế nhất trong ba mô hình: RNN đạt MAE = 2,3233 và RMSE = 3,2098, thấp nhất trong nhóm. Đường dự đoán phản ứng rõ hơn với các thay đổi của giá, nhưng vẫn thấp hơn đáng kể so với thực tế trong giai đoạn tăng mạnh.
- LSTM và Stacked LSTM dự đoán quá thấp khi giá tăng: LSTM đạt MAE = 3,1168; RMSE = 4,2409, còn Stacked LSTM đạt MAE = 3,6917; RMSE = 4,9753. Đường dự đoán của Stacked LSTM phẳng hơn, thể hiện khả năng phản ánh các đợt tăng mạnh còn hạn chế.
- Các biến động đột ngột là điểm yếu chung: Khi giá thực tế tiến gần 90 vào tháng 5/2024, cả ba mô hình đều dự đoán thấp hơn nhiều. Đường dự đoán cũng có biểu hiện trễ và làm mượt biến động. Kết quả cho thấy tăng độ sâu kiến trúc chưa giúp cải thiện độ chính xác trong lần thực nghiệm này.

c. So sánh sai số dự đoán và thời gian huấn luyện của ba mô hình



- RNN có sai số thấp nhất trong nhóm: MAE lần lượt khoảng 2,323; 3,117; 3,692, còn RMSE khoảng 3,210; 4,241; 4,975 đối với RNN, LSTM và Stacked LSTM. Cả hai chỉ số đều cho thứ tự: RNN tốt nhất, tiếp đến LSTM, rồi Stacked LSTM.
- RMSE cao hơn MAE ở cả ba mô hình: RMSE phạt mạnh các sai số lớn, nên sự chênh lệch này phù hợp với những khoảng dự đoán lệch nhiều khi giá tăng đột ngột ở biểu đồ trước. Mô hình cần cải thiện khả năng phản ứng với các biến động lớn.
- RNN huấn luyện nhanh nhất: Thời gian chạy đủ 60 epoch lần lượt khoảng 5,1 giây, 6,6 giây và 11,3 giây. Stacked LSTM mất hơn hai lần thời gian của RNN nhưng có sai số cao hơn, nên chưa mang lại lợi ích tương ứng với chi phí tính toán trong thử nghiệm này

4.1.7.Demo



4.2 Dataset 2

4.2.1. Import và thiết lập tham số

```
import os

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

import time
from pathlib import Path

import kagglehub
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

from IPython.display import display


SEED = 42

TARGET_COL = "Close"

START_DATE = "2016-01-01"
END_DATE = "2023-12-31"

WINDOW_SIZE = 30
BATCH_SIZE = 64
HIDDEN_SIZE = 64

EPOCHS = 60
LEARNING_RATE = 0.001

TRAIN_RATIO = 0.70
```

4.2.2. Chia dữ liệu theo thời gian và chuẩn hóa

```
prices = daily_df[
    ["Price"]
].to_numpy(dtype=np.float64)

n_samples = (
    len(prices) - WINDOW_SIZE
)

n_train = int(
    n_samples * TRAIN_RATIO
)

n_val = int(
    n_samples * VAL_RATIO
)

n_test = (
    n_samples - n_train - n_val
)

if n_train < 1 or n_val < 1 or n_test < 2:
    raise ValueError(
        "Không đủ mẫu để chia train/validation/test."
    )

# Mỗi vị trí là ngày mục tiêu của một cửa sổ.
target_positions = np.arange(
    WINDOW_SIZE,
    len(prices)
)

train_positions = (
    target_positions[:n_train]
)

val_positions = (
    target_positions[
        n_train:n_train + n_val
    ]
)
```

4.2.3. Xây dựng và huấn luyện RNN cơ bản

```
set_seed()

rnn_model = keras.Sequential(
    [
        keras.Input(
            shape=(WINDOW_SIZE, 1)
        ),

        layers.SimpleRNN(
            units=HIDDEN_SIZE,
            activation="tanh",
            return_sequences=False
        ),

        layers.Dense(
            units=1,
            activation="linear"
        )
    ],
    name="AMZN_RNN"
)

rnn_model.summary()

rnn_model = train_model(
    "RNN",
    rnn_model
)
```

4.2.4. Xây dựng và huấn luyện LSTM một tầng

```
# ============================================================
# CELL 10: XÂY DỰNG VÀ HUẤN LUYỆN LSTM MỘT TẦNG
# ============================================================

set_seed()

lstm_model = keras.Sequential(
    [
        keras.Input(
            shape=(WINDOW_SIZE, 1)
        ),

        layers.LSTM(
            units=HIDDEN_SIZE,
            activation="tanh",
            recurrent_activation="sigmoid",
            return_sequences=False
        ),

        layers.Dense(
            units=1,
            activation="linear"
        )
    ],
    name="AMZN_LSTM"
)

lstm_model.summary()

lstm_model = train_model(
    "LSTM",
    lstm_model
)
```

4.2.5. Xây dựng và huấn luyện Stacked LSTM hai tầng

```
# ============================================================
# CELL 11: XÂY DỰNG VÀ HUẤN LUYỆN STACKED LSTM
# ============================================================

set_seed()

stacked_lstm_model = keras.Sequential(
    [
        keras.Input(
            shape=(WINDOW_SIZE, 1)
        ),

        layers.LSTM(
            units=HIDDEN_SIZE,
            activation="tanh",
            recurrent_activation="sigmoid",
            return_sequences=True
        ),

        layers.Dropout(
            rate=0.2,
            seed=SEED
        ),

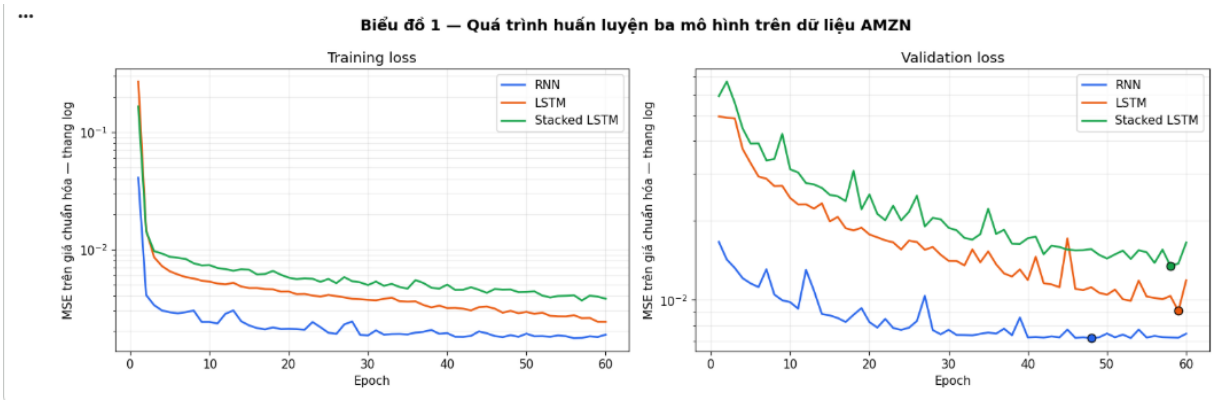
        layers.LSTM(
            units=HIDDEN_SIZE,
            activation="tanh",
            recurrent_activation="sigmoid",
            return_sequences=False
        ),

        layers.Dense(
            units=1,
            activation="linear"
        )
    ],
    name="AMZN_Stacked_LSTM"
)

stacked_lstm_model.summary()
```

4.2.6.Kết quả thực nghiệm

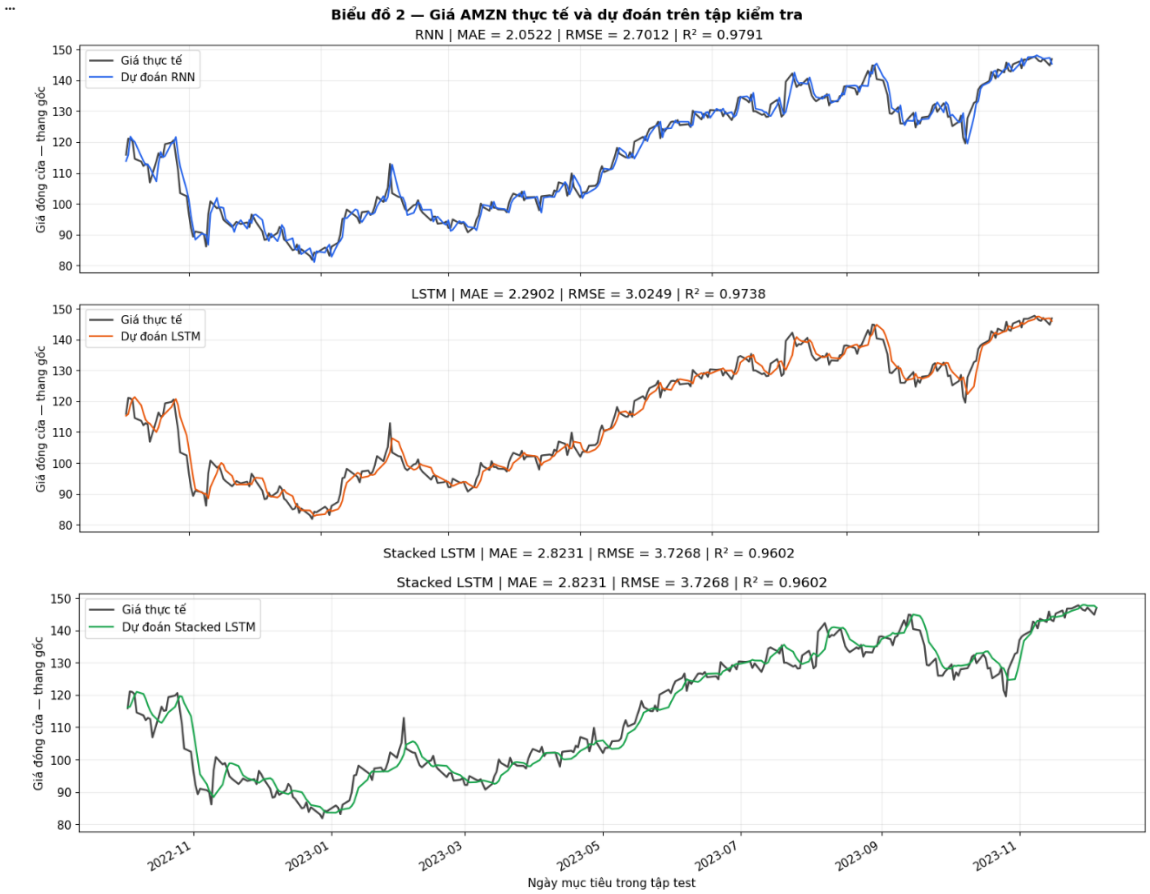
a. So sánh quá trình hội tụ của ba mô hình Keras trên dữ liệu AMZN



- Cả ba mô hình đều học được quy luật của dữ liệu: Training loss giảm mạnh ở những epoch đầu, sau đó tiếp tục giảm chậm đến epoch 60. Trục tung sử dụng thang log nên có thể quan sát rõ sự khác biệt giữa các mô hình ngay cả khi loss đã nhỏ.

- RNN có loss thấp nhất: Đường training loss và validation loss của RNN nằm dưới hai mô hình còn lại trong phần lớn quá trình huấn luyện. Kết quả cho thấy kiến trúc RNN cơ bản phù hợp với cách biểu diễn dữ liệu và cấu hình huấn luyện trong thực nghiệm này.
- LSTM tốt hơn Stacked LSTM về validation loss: Validation loss của LSTM giảm tương đối đều, dù có một số dao động. Stacked LSTM có nhiều đợt tăng giảm hơn và duy trì sai số cao hơn, cho thấy việc bổ sung tầng chưa mang lại cải thiện trong lần chạy này.
- Chưa thấy dấu hiệu quá khớp kéo dài rõ rệt: Training loss giảm trong khi validation loss nhìn chung cũng giảm, thay vì tăng liên tục ở giai đoạn cuối. Các dấu chấm đánh dấu epoch có validation loss thấp nhất; bộ trọng số tại những epoch này được dùng để đánh giá trên test.

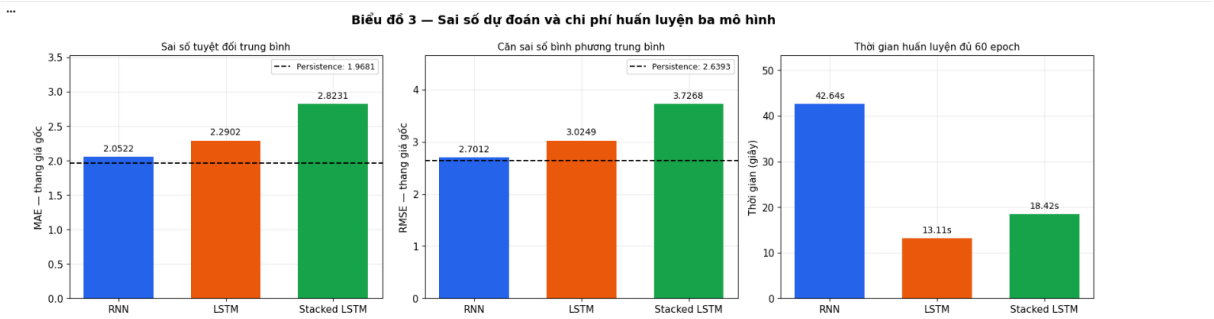
b. So sánh giá đóng cửa AMZN thực tế và dự đoán trên tập kiểm tra



- RNN dự đoán sát nhất trong ba mô hình: RNN đạt MAE = 2,0522; RMSE = 2,7012; R^2 = 0,9791. Đường dự đoán bám khá sát giá thực tế, phản ánh được cả giai đoạn giảm cuối năm 2022 và xu hướng phục hồi trong năm 2023.
- LSTM có kết quả gần với RNN: LSTM đạt MAE = 2,2902; RMSE = 3,0249; R^2 = 0,9738. Mô hình nhận diện tốt xu hướng chung nhưng có độ trễ ở một số điểm đảo chiều, khiến đường dự đoán chưa theo kịp những thay đổi nhanh của giá.

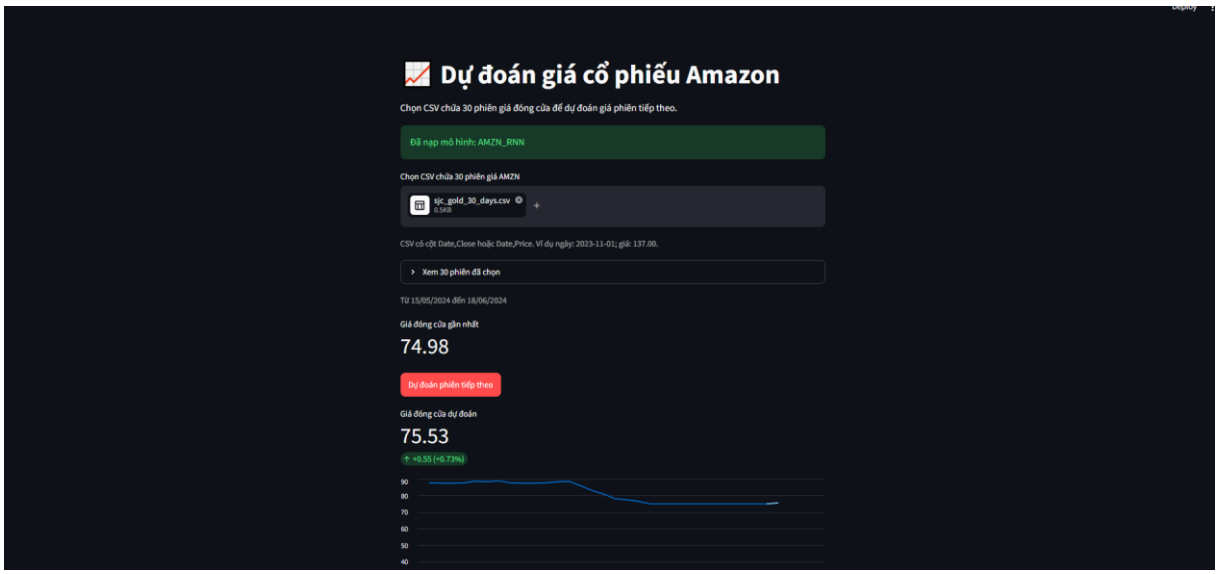
- Stacked LSTM có sai số cao nhất: Mô hình đạt MAE = 2,8231; RMSE = 3,7268; R^2 = 0,9602. Đường dự đoán vẫn phản ánh được xu hướng lớn nhưng làm mượt các biến động mạnh hơn, đặc biệt ở những đoạn giá giảm hoặc tăng đột ngột.
- Khả năng bám sát xu hướng chưa đồng nghĩa dự báo tốt mọi biến động: Các mô hình đều gặp khó khăn tại những điểm thay đổi nhanh. Đây là đánh giá từng phiên tiếp theo bằng 30 giá thực tế trước đó, nên kết quả không thể diễn giải thành khả năng dự đoán liên tục nhiều phiên tương lai từ một cửa sổ ban đầu.

c. So sánh độ chính xác và chi phí huấn luyện của ba mô hình



- RNN dẫn đầu về độ chính xác trong nhóm: RNN có MAE và RMSE thấp nhất, tiếp đến là LSTM rồi Stacked LSTM. Thứ tự này thống nhất với mức độ bám sát đường giá thực tế ở biểu đồ trước.
- LSTM huấn luyện nhanh nhất: Thời gian chạy đủ 60 epoch lần lượt là RNN: 42,64 giây; LSTM: 13,11 giây; Stacked LSTM: 18,42 giây. Trong môi trường thực nghiệm này, RNN mất nhiều thời gian nhất dù có kiến trúc đơn giản hơn; tốc độ còn phụ thuộc cách thư viện thực thi và thiết bị sử dụng.
- LSTM có sự cân bằng tốt giữa thời gian và sai số: So với Stacked LSTM, LSTM một tầng vừa huấn luyện nhanh hơn, vừa có sai số thấp hơn. RNN đạt độ chính xác cao hơn LSTM nhưng cần thời gian huấn luyện lớn hơn đáng kể.
- Cả ba mô hình vẫn chưa vượt qua Persistence: Phương pháp lấy giá phiên gần nhất làm dự đoán đạt MAE = 1,9681; RMSE = 2,6393, thấp hơn cả RNN. Khoảng cách giữa RNN và Persistence khá nhỏ, nhưng kết quả hiện tại vẫn chưa chứng minh được lợi thế của ba mô hình so với phương pháp đơn giản này.

4.2.7 Demo trên mô hình tốt nhất


Deploy
Dự đoán giá cổ phiếu Amazon
Chọn CSV chứa 30 phiên giá đóng cửa để dự đoán giá phiên tiếp theo.
Đã nạp mô hình: AMZN_RNN
Chọn CSV chứa 30 phiên giá AMZN
sjc_gold_30_days.csv
0.5KB
CSV có cột Date,Close hoặc Date,Price. Ví dụ ngày: 2023-11-01; giá: 137.00.
Xem 30 phiên đã chọn
Từ 15/05/2024 đến 18/06/2024
Giá đóng cửa gần nhất
74.98
Dự đoán phiên tiếp theo
Giá đóng cửa dự đoán
75.53
↑ +0.55 (+0.73%)
90
80
70
60
50
40